MBS-fast

Руководство пользователя и разработчика

Геометрическая и физическая оптика для многогранных частиц

Документ описывает физическую модель, численные алгоритмы, сборку, параметры запуска, форматы данных, ускорение на CPU и GPU, а также обязательные проверки точности. В командах используются канонические имена параметров из текущего интерфейса программы; устаревшие сокращения указаны отдельно и нужны только для воспроизводимости старых сценариев.

Редакция от 12 августа 2026 г.

Документ для актуального интерфейса командной строки

О руководстве

MBS-fast предназначен для расчёта дальнего рассеяния света многогранными частицами. Геометрическая стадия строит и трассирует полигональные пучки, перенося комплексную матрицу Джонса через отражения и преломления. В режиме физической оптики (PO) каждый выходной пучок дополняется дифракционным интегралом по его апертуре. После когерентного суммирования амплитуд результат переводится в матрицу Мюллера и, при необходимости, усредняется по ориентациям частицы.

Руководство рассчитано на два способа чтения. Для первого запуска достаточно разделов «Быстрый старт», «Сборка и исполняемые файлы», «Параметры командной строки» и «Обязательные проверки результата». Разделы о трассировке, коэффициентах Френеля, дифракционном интеграле и реализации на CPU/GPU предназначены для проверки физики и модификации кода. Приложения содержат формулы метрик и контрольный список для воспроизводимого расчёта.

Содержание

О руководстве i

I Практическая работа 1

1 Fullauto: запуск, математика и проверка результата 1

2 Быстрый старт 12

II Физическая модель и геометрия 13

3 Назначение программы 13

4 Координатные системы 15

5 Трассировка лучей 16

6 Коэффициенты Френеля 18

7 Матрицы Джонса и диады 19

8 Переход к матрице Мюллера 21

9 Дифракция в приближении физической оптики 23

10 Обрезка пучков в невыпуклых частицах и агрегатах 25

11 Размер частицы и масштабирование геометрии 28

III Ориентации и угловые сетки **31**

12 Ориентационное усреднение **31**

13 Угловая сетка рассеяния **43**

IV Реализация и производительность **45**

14 Реализация на CPU и AVX **45**

15 Реализация на GPU **47**

V Сборка, запуск и данные **54**

16 Сборка и исполняемые файлы **54**

17 Запуск и первичная диагностика **56**

18 Параметры командной строки **57**

19 Выходные файлы и интегральные величины **61**

VI Точность, проверка и воспроизводимость **62**

20 Обязательные проверки результата **62**

21 Оценка погрешности и выбор параметров **63**

22 Ускорения и контролируемые приближения **66**

23 Диагностика и воспроизводимость **69**

VII Приложения **74**

A Связь с файлами кода **74**

B Контрольные формулы для отчётов **75**

C Контрольный список перед публикацией результатов **76**

D Калибровка и многоуровневое ускорение GPU **77**

Часть I

Практическая работа

1. Fullauto: запуск, математика и проверка результата

Руководство к объединённому коду для подготовки MBS-fast 2.0, 26 октября 2026. Источники поведения: `src/FullAuto*.inc`, `src/FullAuto.cpp`, `src/CliOptions.cpp`. Проценты в примерах относятся к численной погрешности внутри модели PO или GO.

1.1. Самый короткий запуск

```
make -C cpu -j2
cpu/bin/mbs_po_mpi --method po --backend cpu \
  --particle 1 5 2 --ri 1.3116 0 --wavelength-um 0.532 \
  --fullauto 0.02 --output results/column_5um
```

`po` — физическая оптика; `cpu` — вычисление на процессоре. `--particle 1 L D` задаёт правильный шестигранный столбик: L — длина вдоль его оси, D — диаметр основания. Здесь L=5 и D=2 мкм. `--ri REAL IMAG` задаёт комплексный показатель преломления. Длина волны и координаты частицы должны иметь одинаковые единицы. `0.02` — допуск 2%. Каталог `-\/-output` хранит матрицы, параметры и восстановление.

Не нужно добавлять Sobol, число φ, стартовые ориентации, коэффициенты аналитической модели и отсечение только ради включения автоматики: контроллер задаёт их сам. Это один процесс контроллера; не запускайте несколько MPI-контроллеров в одну папку.

Частица из файла с максимальным габаритом Dmax=5 мкм:

```
cpu/bin/mbs_po_mpi --method po --backend cpu \
  --particle-file examples/cube.particle --resize-dmax-um 5 \
  --ri 1.3116 0 --wavelength-um 0.532 \
  --fullauto 0.02 --output results/file_5um
```

Dmax — максимальное расстояние между вершинами. Оно отличается от длины столбика, его диаметра основания и диаметра равновеликого шара. Данные о симметрии в файле не считаются доказательством: сначала используется полная область ориентаций.

1.2. GPU, конусы и подробная команда

Сборка FP64 для V100: `make -C gpu fp64 GPU_ARCH=70 -j2`. Для RTX 3080 Ti используйте архитектуру 86, для RTX 4090 — 89. Сборка `fp32` быстрее на потребительских картах, но её арифметическую погрешность нужно проверить отдельно: статистический интервал Sobol её не измеряет.

```
# Вперёд: 0-25 градусов, один GPU.
CUDA_VISIBLE_DEVICES=0 gpu/bin/mbs_po_gpu_double \
  --method po --backend cuda --particle 1 100 50 \
  --ri 1.3116 0 --wavelength-um 0.532 \
  --fullauto 0.02 --fullauto-theta-range 0 25 \
  --output results/column_100_forward

# Назад: 170-180 градусов, два GPU одного контроллера.
CUDA_VISIBLE_DEVICES=0,1 gpu/bin/mbs_po_gpu_double \
  --method po --backend cuda --particle 1 100 50 \
  --ri 1.3116 0 --wavelength-um 0.532 \
  --fullauto 0.02 --fullauto-theta-range 170 180 \
  --fullauto-gpus 0,1 --output results/column_100_backward
```

Номера GPU отсчитываются после применения `CUDA_VISIBLE_DEVICES`. Для двух независимых размеров запускайте два контроллера с разными каталогами и непересекающимися списками GPU. Флаги старого multisize не совмещаются с fullauto.

Расширенный пример намеренно фиксирует часть параметров:

```
CUDA_VISIBLE_DEVICES=0 gpu/bin/mbs_po_gpu_double \
  --method po --backend cuda --particle-file examples/cube.particle --resize-dmax-um 5 \
  --ri 1.3116 0 --wavelength-um 0.532 \
  --fullauto 0.02 --fullauto-theta-range 170 180 \
  --max-reflections 8 --phi-points 4 \
  --fullauto-initial 512 --fullauto-min-correction 256 \
  --max-orientations 1048576 --threads 4 \
  --fullauto-rotation-symmetry off --fullauto-mirror off \
  --cutoff-profile off --fullauto-analytic-controls off \
  --output results/fixed_settings
```

Флаг	Что меняет
`--max-reflections 8`	Фиксирует целевую глубину. Автопоиск 6...18 выключен. Финальная проверка может обнаружить недостаточную глубину.
`--phi-points 4`	Фиксирует число наблюдательных азимутов; поиск φ не заменит это значение.
`--fullauto-initial 512`	Начальное N на один seed, а не окончательное N.
`--fullauto-min-correction 256`	Минимальное число ориентаций поправки на один seed.
`--max-orientations 1048576`	Верхняя граница N; достижение границы не означает сходимость.
`--threads 4`	Явное число потоков; автоматический подбор не должен его переопределять.
`--fullauto-rotation-symmetry off`	Не искать сокращение области по вращениям тела.

Флаг	Что меняет
`--fullauto-mirror off`	Не использовать зеркальное сокращение.
`--cutoff-profile off`	Фиксировать расчёт без отсечения; автоматический balanced с поправкой не включается.
`--fullauto-analytic-controls off`	Вычислять полную PO без аналитических контрольных компонент.

Обычно короткая команда удобнее. Расширенный вариант нужен для воспроизводимых сравнений и проверки одной настройки. Явное ограничение имеет приоритет, но несовместимые режимы должны завершаться ошибкой, а не молча игнорироваться.

1.3. Что означает требуемая точность

M — матрица Мюллера 4×4; она переводит падающий вектор Стокса в рассеянный. ϑ — угол рассеяния: 0° вперёд, 180° назад. Для каждого проверяемого ϑ контролируются все 16 элементов. Масштаб абсолютной ошибки Mij — M11 того же угла. Поэтому нуль M12 не приводит к делению на M12. Дополнительно по умолчанию проверяются отношения Mij/M11; их погрешность задаётся в абсолютных долях отношения.

Для каждого элемента и угла складываются его статистическая оценка, его оценка интерполяции и фиксированные резервы. Максимум берётся **после** этого сложения:

$$E = \max_{\theta,j}\{e_{\mathrm{stat},\theta j} + e_{\mathrm{interp},\theta j} + r_{\mathrm{fixed}}\}.$$

Отдельно проверяются непосредственно вычисленная и полная выходная матрицы. Нельзя складывать максимальные статистическую и интерполяционную ошибки разных элементов и выдавать это за реализованный критерий. Интерполяционный допуск равен EPS/4: для 2% это 0,5%, для 1% — 0,25%. Резервы зависят от включённых проверок глубины, симметрии, аналитики и исполнения. Фактические значения записаны в файлах параметров и `pointwise_budget_production.csv`.

95%-интервалы здесь поточечные и статистические. Они не гарантируют одновременно 95% для всех углов, не доказывают отсутствие узкого пика между проверками и не измеряют ошибку самой модели PO относительно точного электромагнитного решения. Отключённая проверка обозначается как непроверенная, а не как нулевая ошибка.

1.4. Последовательность работы

1. Проверить аргументы, геометрию, единицы и доступные ресурсы. Построить запрошенную сетку вывода и стартовую оценку N. Если есть совместимый чекпоинт, восстановить его.
2. Подобрать целевую глубину внутренних взаимодействий, если она не задана. На CUDA перед этим проводится короткая настройка исполнения и φ для пилота; полный подбор выполняется после выбора глубины.

3. Проверить допустимые симметрии. Обучить аналитические контрольные коэффициенты и проверить их на других seed. Выбрать φ и параметры исполнения по стоимости достижения допуска, а не только по времени одного малого запуска.
4. Вычислять опорные и проверочные ϑ, восемь независимых реплик и уровни поправок. Сохранять завершённые блоки; использовать прежние блоки для совпадающих запросов.
5. Оценить статистику, остаток интерполяции и устойчивость префиксов. Уточнять ориентации или ϑ там, где это требуется. Шумный остаток сначала получает больше парных образцов; новый опорный ϑ добавляется при подтверждённой структуре.
6. Опубликовать промежуточную матрицу и оценки. После прохождения рабочих критериев выполнить включённые независимые и физические проверки и сохранить итог.

`--fullauto-final-checks off` отключает финальные перепроверки. Это меняет объём проверенного результата, а не делает пропущенные проверки успешными. По умолчанию их сохраняют. Такой режим нельзя использовать с GO balanced, которому обязательна проверка ошибки отсечения. При явном `-\/-convergence-passes` число проходов сохраняется.

1.5. Стартовое N и глубина n

N — число ориентаций **на одну** Sobol-реплику и уровень. Восемь реплик с N=512 означают 4096 ориентаций базового уровня; поправки имеют свои N. n — максимальная глубина внутренних взаимодействий в трассировке.

По умолчанию coherent PO использует эмпирическую оценку по уже завершённым расчётам, размеру, длине волны, диапазону ϑ, форме и допуску. Это только начало, а не предсказанная гарантия сходимости. В реализации:

$$\begin{aligned} s = {} & 8.5128934 + 2.0264267 \log_2(1 + L_{\max}/(8\lambda)) \\ & + 0.6416260\, I(\theta_{\max} > 90^\circ) + 4.0403518\, I(\Delta\theta > 90^\circ) \\ & + 0.8222766\, I(\text{controls off}) - 2.7035215\, I(\text{не столбик типа 1}). \end{aligned}$$

$$N_{\text{forecast}} = 2^s (0.01/\text{EPS})^2, \qquad N_0 = \text{pow2up}(\max(512, N_{\text{forecast}}/16)) \,.$$

Lmax — наибольшее ребро в оценке геометрии; I — индикатор условия (0 или 1); pow2up — округление вверх до степени двойки. До округления применяется предел, связанный с `-\/-max-orientations`. Коэффициенты модели получены по 234 записям, 73 уникальным когерентным конфигурациям; перенос на новую форму проверяется последующей сходимостью. Формула со знаком «не столбик» отражает эти данные, а не общий физический закон для произвольных частиц.

После углового пилота эта оценка становится верхним предложением для первого производственного раунда. По измеренному времени контроллер выбирает степень двойки с ориентиром 1800 секунд работы на одну GPU; при отсутствии измерений начинает с 512 на seed. Уже готовые полные префиксы восьми seed и минимальные бюджеты поправок сохраняются. `initial_production_work.json` содержит предложенное и выбранное число ориентаций, прогноз времени и признак наличия измерений. Это ориентир стоимости, а не гарантия завершения за 30 минут. Решение принимается без чтения значений производственной матрицы. Явный `-\/-fullauto-initial`, загруженная адаптивная конфигурация и `--fullauto-local`

`off` сохраняют заданный старт. Следующие раунды уточняются по ошибке; все критерии приёмки остаются прежними. Ранняя первая оценка не означает, что расчёт сошёлся.

`--fullauto-initial-model xi` возвращает старый ориентир beta×gamma по мелкому дифракционному шагу; он часто завышает старт. Явный `-\/-fullauto-initial` заменяет обе модели. Для GO и некогерентной PO применяется другой ограниченный старт, связанный с оптической толщиной поглощения; GO без поглощения масштабно инвариантна.

Поиск n начинается с 6 и может поднять его до 18. Сравниваются n и n+2 на общих ориентациях: сначала уменьшается шум разности, затем решается, нужно ли увеличивать n. Старт пилота по умолчанию min(512,N0), независимый от большого производственного N. Параметры: `-\/-fullauto-depth-start`, `-\/-fullauto-depth-max`, `-\/-fullauto-depth-pilot`, `-\/-fullauto-depth-order`. Неразрешённый пилот или достигнутый предел глубины не принимаются как успех. `--max-reflections N` отключает поиск целевого n, но не обещает прохождение включённого финального сравнения n/n+2.

1.6. Усреднение Sobol и три угла Эйлера

Поворот задаётся R=Rz(α)Ry(β)Rz(γ). Для равномерного случайного положения частицы нужна мера Хаара — равномерная мера на группе трёхмерных поворотов SO(3):

$$\langle M\rangle = \frac{1}{8\pi^2}\int_0^{2\pi} d\alpha \int_0^{\pi} \sin\beta\, d\beta \int_0^{2\pi} M(R,\theta)\, d\gamma.$$

Для полной области точки u,v,w из единичного куба превращаются в cosβ=1−2u, γ=2πv, α=2πw. Равномерно раскладывать β без веса sinβ было бы неверно. Sobol строит последовательность с равномерным покрытием куба; вложенное перемешивание Owen с seed делает независимые рандомизированные реплики. Это не независимые случайные точки внутри каждой реплики и не декартова сетка Nα×Nβ×Nγ. Поэтому сравнение одного N Sobol с произведением трёх сеточных размеров не означает такое же увеличение точности или ускорение.

Восемь seed используются для оценки неопределённости. Пусть Yr — целая оценка матрицы из реплики r, включая связанные поправки. Тогда

$$\bar{Y} = \frac{1}{8}\sum_{r=1}^{8} Y_r, \quad s_Y^2 = \frac{1}{7}\sum_{r=1}^{8}(Y_r-\bar{Y})^2, \quad h_{95} = t_{0.975,7}\, s_Y/\sqrt{8}.$$

t — квантиль распределения Стьюдента, около 2,365. Для Mij/M11 учитывается ковариация числителя и знаменателя и проверка разрешимости отношения; малый шумный знаменатель не разрешается заменой на удобный нулевой интервал. Данные половинных префиксов дополнительно проверяют устойчивость.

N растёт степенями двойки. Уже рассчитанный префикс используется повторно; `-\/-sobol-offset` в рабочих заданиях задаёт его продолжение. Производственные, обучающие и проверочные seed разделены. Один seed не заменяет восемь реплик.

Sobol — наиболее практичный текущий вариант для наших неоднородных PO-интегралов и локального уточнения: можно наращивать префикс и совместно считать разности уровней. Это не доказательство превосходства над Лебедевым или разложением Вигнера на любой форме. Их преимущества зависят от гладкости и симметрии конкретной задачи.

1.7. Зачем φ, если уже есть α

α вращает частицу вокруг направления падающего света. φ задаёт азимут наблюдателя. При полном усреднении по α дополнительные φ могут снижать дисперсию, сохраняя математическое ожидание. В каждой ориентации это разные операции; для поляризационных элементов также нужен корректный поворот базиса Стокса.

`-\/-haar-alpha` добавляет третью координату Sobol. Fullauto PO использует её сам. По мере увеличения числа ориентаций не требуется автоматически делать φ столь же мелким, как в старой двухугловой сетке. Но φ=1 не гарантирует минимальное время: дополнительные направления наблюдения используют уже трассированные пучки. Контроллер выбирает компромисс «стоимость × дисперсия» и может уточнять φ локально. `--phi-points N` закрепляет правило. `--sobol-seed 32768 SEED` относится к ручному запуску; в fullauto используйте `--fullauto-initial 32768` для начального N.

Вращательная и зеркальная симметрии относятся к телу и проверяются отдельно. Шестикратная симметрия столбика не даёт права делить лабораторное α или φ на шесть. Для произвольной частицы без подтверждённой симметрии остаётся полная область.

1.8. Что вычисляется аналитически

1.8.1. Дифракция отдельного пучка

Плоский многоугольный выходной пучок даёт интеграл вида

$$F(\mathbf{q}) = \int_A \exp(i\mathbf{q}\cdot\mathbf{x})\, dA.$$

Для постоянной амплитуды на плоской апертуре его можно свести к сумме по рёбрам. При малой фазе и почти нулевых знаменателях используются устойчивые пределы и моменты. Это аналитическое вычисление дифракции **для данной ориентации**; оно не выполняет всего ориентационного усреднения. Комплексные амплитуды пучков складываются когерентно до перехода к матрице Мюллера. В некогерентном режиме складываются отдельные интенсивности: это другая физическая постановка.

1.8.2. Аналитические контрольные компоненты

Пусть C(R,ϑ) — дешёвая физическая модель с известным средним μC(ϑ), а M — полная PO. Контрольная переменная даёт оценку

$$\widehat{M} = \overline{M - bC} + b\mu_C.$$

b — коэффициент уменьшения дисперсии, обученный отдельно от производственной выборки. При корректном μC вычитание и возврат среднего сохраняют ожидание. Внутренние пути, затенение и интерференция, отсутствующие в C, остаются в M−bC. Поэтому усреднить коэффициент Френеля и заменить им все отражения недостаточно.

Контроли включают самовклады отражающих граней и модель тени. Для M11 используются их физические средние; поляризационные контроли и поворотные компоненты проходят собственную проверку. Неподходящие компоненты отключаются после проверки по полной PO: это допустимый отказ от ускорения, а не отказ от расчёта рассеяния.

1.8.3. Моменты Френеля и фазовые средние

μ — косинус угла падения на грань, rs(μ), rp(μ) — коэффициенты Френеля двух поляризаций. Отражательная способность R(μ)=(|rs|²+|rp|²)/2 зависит от μ. Момент — интеграл этой функции с весом; например,

$$c_\ell = \frac{2\ell+1}{2}\int_0^1 R(\mu)P_\ell(\mu)\,d\mu,$$

где P_ℓ — полином Лежандра. В физических ядрах присутствуют также геометрические и проекционные веса. Теорема сложения и разложение плоской волны позволяют аналитически усреднить угловые фазовые множители, используя сферические функции Бесселя. Общие моменты повторно используются для многих ϑ. Гладкие одномерные интегралы Френеля, таблицы радиальных ядер и часть интегралов по рёбрам остаются численными. Это аналитико-численный контроль, не замкнутое решение всех путей.

Настройка	Смысл
`-\/-analytic-facet-average`	Явно запросить аналитический контроль граней; fullauto умеет включать и проверять его автоматически.
`--analytic-shadow-control circular`	Контроль тени круглой апертурой с геометрически определённой площадью. Отличие от реальной формы остаётся в остатке.
`--analytic-mean-cache FILE`	Файл кэша средних для совпадающих параметров. В fullauto этот параметр можно оставить автоматике.
`--analytic-mean-threads N`	Потоки подготовки аналитического среднего; явное значение фиксируется.
`--fullauto-analytic-controls off`	Полная численная PO, без этих контролей.

Для большой частицы подготовка среднего может стоить заметного времени и RAM. Кэш экономит повторную подготовку; это не дополнительный физический эффект. Режим GO и некогерентный режим не используют те же когерентные PO-контроли.

1.9. Поправки по глубине и отсечению

Несколько уровней используют тождество

$$\mathbb{E}[M_{n_L}] = \mathbb{E}[M_{n_0}] + \sum_{\ell=1}^{L}\mathbb{E}[M_{n_\ell} - M_{n_{\ell-1}}].$$

Это многоуровневая оценка: дорогие разности могут иметь меньшую дисперсию, чем полная матрица, и потому требовать меньше ориентаций. Оба слагаемых разности используют одинаковые ориентации. Где поддерживается, общая трассировка сохраняет результаты для

двух глубин. Разность берётся между матрицами полных когерентных решений соответствующей глубины, а не между интенсивностями отдельно отброшенных лучей. Ковариации учитываются через полную сумму внутри каждой из восьми реплик.

По умолчанию когерентная PO использует balanced с парной поправкой:

$$\widehat{M} = \overline{B}_N + \overline{(O-B)}_m.$$

B — расчёт с отсечением, O — без отсечения. Ориентации O и B в поправке совпадают. Физическая цель остаётся O. Небольшой m экономит работу, пока дисперсия поправки достаточно мала; в коде используются минимальные блоки и доля префикса. Отдельный подбор пресетов по умолчанию больше не нужен. Время всё равно зависит от числа пучков поправки и её дисперсии: гарантированного ускорения нет.

Явный `--cutoff-profile balanced` фиксирует пресет и не эквивалентен автоматической схеме с O−B. Явные пороги также имеют приоритет. Для GO balanced применяется отдельная проверка смещения относительно off, описанная в `GO_BALANCED.md`.

1.10. ϑ, интерполяция и короткие раунды

Выходная сетка, опорные ϑ и проверочные ϑ — разные множества. Без внешней сетки fullauto строит достаточно плотную сетку вывода из размера, λ и диапазона. Прямой расчёт производится на адаптивном подмножестве. `--theta-grid-file FILE` фиксирует именно углы вывода, но сам по себе не означает прямое вычисление каждой строки.

Локальная кубическая интерполяция I линейна по значениям матрицы при фиксированных узлах. Проверяется остаток $\Delta = M(\theta) - I[M_{\text{support}}](\theta)$ на одинаковых ориентациях. Это позволяет сокращать общий ориентационный шум. Бюджет остатка теперь отдельный: он не наследует максимальное N соседнего дорогого угла. При различных рабочих φ для остатка обе стороны используют единое правило φ. Основной расчёт сохраняет свои правила. Выборка остатка фиксируется до чтения значений, а затем уточняется по неуспешной проверке.

Новый опорный угол начинается с обычного стартового бюджета. Полные уже готовые префиксы восьми seed сохраняются. Планировщик ограничивает объём следующего раунда по измеренной стоимости; ориентир 1800 рабочих секунд на GPU не является жёстким сроком. Неделимый префикс, трассировка или чтение кэша могут занимать больше.

Важная проверка на реальных данных: для столбика 794,3×196,2 мкм, λ=0,532, m=1,3116, участка 170,042–170,072°, восьми seed и N=512 на уровень остаточная оценка составила 322,691% при смешанных φ=4/64 и 0,0329171% при общем φ=64. Это снижение шума **проверочного остатка на этом участке**, не ошибка всей матрицы и не измеренное ускорение всего расчёта. Уровни 14/16/18 считались без отсечения.

В полном DAT проверочный угол также может содержать интерполяцию, хотя для него есть отдельный прямой расчёт. Для различения используйте `value_source` и `point_type` в аннотированном CSV и `mueller_evaluated.dat`. Не заменяйте одни значения другими постфактум без новой проверки принятого выходного оценивателя.

1.11. Логи, результат и восстановление

При восстановимой ошибке бюджета RAM постоянный рабочий процесс освобождается, а незавершённый блок повторяется один раз в новом процессе на тех же ориентациях. Частичный результат не принимается; исходный журнал сохраняется как `stdout.memory-retry.log`. Если сработал системный резерв RAM, контроллер ждёт его восстановления не более 60 секунд после освобождения рабочего процесса. Ожидание и повтор отражаются в `resource_wait.json` и журнале событий. Резерв не уменьшается. Повторная нехватка памяти или другая вычислительная/CUDA-ошибка останавливает задачу с сохранением завершённых чекпоинтов.

Файл	Назначение
`fullauto_status.json`	Статус, параметры и последняя опубликованная оценка. Только `converged` означает прохождение включённых критериев.
`intermediate_current.json`	Ссылка на последний снимок матрицы; подходит для просмотра незавершённого расчёта.
`round_snapshots/`	Матрицы и точности предыдущих опубликованных раундов.
`parameters_accuracy.csv`	Производственные N по уровням, φ, оценки элементов и отношений в снимке.
`interpolation_orientation_budget.csv`	Отдельные N проверочного остатка в снимке.
`mueller_fullauto.dat`	Принятая или опубликованная полная сетка; интерпретировать вместе со статусом.
`mueller_evaluated.dat`	Прямые вычисления текущего оценивателя.
`mueller_fullauto_annotated.csv`	Происхождение строк и оценки ошибок. Пустое поле не означает нулевую ошибку.
`execution_progress.json`	Текущие PID, GPU, пакеты, времена и число сохранённых заданий. Это не новая оценка точности.
`fullauto_history.jsonl`	История подбора и исполнения.
`controller_command.json`	Воспроизводимая команда.
`jobs/`, чекпоинты и индексы	Повторное использование завершённой работы. Сохраняйте каталог целиком.

```
python3 -m json.tool results/column_5um/fullauto_status.json
python3 -m json.tool results/column_5um/intermediate_current.json
python3 -m json.tool results/column_5um/execution_progress.json
```

После отключения повторите исходную команду тем же бинарником и в ту же папку. Завершённые блоки загружаются; незавершённый блок может повториться. Чекпоинт версии 6 хранит производственный и остаточный бюджеты, а также φ остатка. Поддержано чтение

старых схем, но совместимость бинарников проверяется отдельно. Нельзя вручную подменять хэш, редактировать готовые матрицы или переносить только один TSV чекпоинта без зависимых блоков.

Для нового бинарника `--fullauto-resume-compatible OLD_BINARY` допустим только при подтверждённой совместимости конкретной пары бинарников и вычислительных профилей. Сам факт успешной сборки такой совместимостью не является. Для нового допуска или окна используйте документированный режим `--fullauto-refine-window A B EPS` и сохранённую исходную конфигурацию; не редактируйте физические параметры существующего каталога.

Служба Linux может автоматически возобновлять процесс после перезагрузки:

```
scripts/run_fullauto_service.sh mbs-column cpu/bin/mbs_po_mpi \
  results/column_service.log --method po --backend cpu \
  --particle 1 5 2 --ri 1.3116 0 --wavelength-um 0.532 \
  --fullauto 0.02 --output results/column_service
```

Этот скрипт устанавливает пользовательскую службу systemd и запускает её. Он управляет процессом; численные решения остаются внутри программы.

1.12. Память, загрузка GPU и границы метода

CPU трассирует и подготавливает пучки; CUDA вычисляет дифракцию и поддержанные части трассировки. Постоянные рабочие процессы сохраняют CUDA-контекст и ограниченные кэши. Готовые пакеты распределяются между GPU, проверка и сохранение данных перекрываются с дальнейшей работой. Видеопамять занята пучками, угловыми буферами, матрицами и рабочими областями; RAM — геометрией, подготовкой и кэшем блоков.

Загрузка GPU может падать при трассировке, сборке префиксов, проверке и I/O. 100% загрузки не является критерием ни правильности, ни минимального времени. Большой пакет снижает накладные расходы, но повышает память и задержку публикации. Контроллер ограничивает память и объём работы; OOM или исчерпание диска не являются успешным завершением. Текущее время задания видно отдельно от возраста оценки.

1.13. GO и extinction-only

```
cpu/bin/mbs_po_mpi --method go --backend cpu \
  --particle 1 100 50 --ri 1.3116 0 --wavelength-um 0.532 \
  --fullauto 0.02 --output results/column_go
```

GO — геометрическая оптика: α усредняется аналитически; конечные угловые ячейки считаются напрямую, кубической PO-интерполяции нет. Полюсные дельта-вклады сохраняются отдельно в `mueller_forward.dat` и `mueller_backward.dat`; это интегральные массы, не значения гладкой диаграммы в градусной точке.

GO mirror по умолчанию выключен. Явное включение поддерживает непоглощающие встроенные типы 1 и 10 с допустимыми симметриями; зеркальная пара восстанавливается через $(M+PMP)/2$, $P=\mathrm{diag}(1,1,-1,-1)$, включая полюсные вклады. Это не простое удвоение числа симметрий. Файловые формы и поглощение не принимаются как поддержанные. Подробности: `GO_MIRROR.md` и `GO_BALANCED.md`.

`-\/-extinction-only` — отдельный CPU-режим PO с Hammersley для выпуклой геометрии. Он вычисляет экстинкцию по оптической теореме без угловой сетки матриц; не является

fullauto и не даёт полного интеграла рассеяния или точного альбедо. Примеры: `EXTINCTION_ONLY.md`.

2. Быстрый старт

2.1. Сборка проверочного бинарного файла

Для воспроизводимой проверки сначала собирают CPU-версию и запускают матрицу коротких тестов интерфейса:

```
make -C cpu -j
make test_release
cpu/bin/mbs_po_mpi --version
```

GPU-версия двойной точности собирается отдельной целью. Именно её следует использовать для проверки точек 0° и 180° и для сравнения с CPU:

```
make -C gpu fp64 -j
gpu/bin/mbs_po_gpu_double --version
```

2.2. Первый расчёт

Следующая команда считает гексагональный столбик длиной 100 мкм и диаметром 70 мкм при $\lambda = 0.532$ мкм. Сетка ориентаций и углов рассеяния строится из дифракционной оценки; широтная сетка уменьшает избыточное число азимутальных узлов около полюсов.

```
CUDA_VISIBLE_DEVICES=0 \
gpu/bin/mbs_po_gpu_double \
  --method po \
  --particle 1 100 70 \
  --refractive-index 1.3116 0 \
  --wavelength-um 0.532 \
  --max-reflections 8 \
  --diffraction-sampling 2 \
  --latitude-phi-grid \
  --backend cuda --threads 16 \
  --cutoff-profile off \
  --output results/column_L100 --close
```

Это контрольная, а не самая быстрая конфигурация. Профили отсечения, FFT и многопроцессорное распределение включают только после сравнения с таким прямым расчётом на нескольких характерных размерах.

2.3. Что проверить после завершения

Расчёт можно использовать дальше, только если одновременно выполнены следующие условия:

1. процесс завершился с нулевым кодом, а выходной файл содержит все строки по θ от 0° до 180°;
2. в логе нет сообщения о достижении ограничения числа пучков или верхней границы адаптивного поиска;
3. отчёт интегральных величин не содержит статуса `check_csca_integration`;

4. более мелкая сетка по ориентациям и углам меняет требуемые элементы матрицы Мюллера меньше заданной погрешности;
5. для новой формы хотя бы одна небольшая задача совпадает между CPU и GPU двойной точности.

Завершившийся процесс ещё не означает сошедшийся физический результат. Глубина трассировки, угловая сетка, ориентационное усреднение, отсечения слабых пучков и численная точность проверяются независимо.

Часть II

Физическая модель и геометрия

3. Назначение программы

MBS-fast решает задачу дальнего рассеяния на частице с плоскими гранями. В отличие от метода дискретных диполей программа не заполняет объём частицы расчётными узлами. Она строит геометрические пучки, возникающие после отражений и преломлений на гранях. Каждый пучок хранит полигон апертуры, направление распространения, оптический путь и комплексную матрицу Джонса. После трассировки для каждого выходного пучка вычисляется дифракционный интеграл физической оптики. Амплитуды суммируются когерентно и преобразуются в матрицу Мюллера.



CPU подготавливает лучи и пучки; CPU или GPU вычисляет дифракцию на сетке направлений

Рис. 1: Последовательность расчёта: геометрия частицы, трассировка, коэффициенты Френеля и матрицы Джонса, дифракция, когерентная сумма, матрица Мюллера и усреднение по ориентациям.

3.1. Физические уровни модели

В расчете используются четыре уровня приближения.

Таблица 4: Физические уровни модели MBS-fast.

Уровень	Что считается	Где появляется в коде
Геометрическая оптика	Пучки отражаются и преломляются на плоских гранях по законам Снеллиуса; малозначимые ветви могут быть отсечены заданным критерием.	`src/scattering`, `Splitting.cpp`.
Френель и Джонс	Поляризационная амплитуда каждой ветви хранится как комплексная матрица Джонса.	`Beam::J`, `Splitting.cpp`.
Физическая оптика	Выходной пучок создаёт дальнее поле, вычисляемое интегралом по его полигональной апертуре.	`HandlerPO::ApplyDiffraction*`, ядра CUDA.
Ориентационное усреднение	Для ансамбля случайно ориентированных частиц вычисляется взвешенная сумма матриц Мюллера по квадратурной или квазислучайной сетке.	`HandlerPOTotal`, `TracerPOTotal`.

В режиме PO пучки одной ориентации по умолчанию суммируются когерентно на уровне амплитуд Джонса. Только после этой суммы строится матрица Мюллера. Параметр `--incoherent` меняет физическую постановку: матрицы Мюллера отдельных пучков складываются без интерференционных членов.

3.2. Обозначения

Таблица 5: Базовые обозначения, используемые дальше.

Символ	Смысл
λ	Длина волны в микрометрах, задаётся параметром `--wavelength-um`.
$k = 2\pi/\lambda$	Волновое число во внешней среде.
$m = n + \mathrm{i}\kappa$	Комплексный показатель преломления частицы относительно внешней среды, задаётся параметром `--refractive-index Re Im`.
$\mathbf{d}$	Направление распространения текущего луча или пучка.
$\mathbf{n}$	Внешняя нормаль грани. В коде знак выбирается так, чтобы ветвление было устойчивым для текущей стороны грани.

Символ	Смысл
θ, ϕ	Углы направления наблюдения в дальней зоне.
$\mathbf{s}(\theta, \phi)$	Единичный вектор направления наблюдения.
$\mathbf{J}$	Комплексная матрица Джонса 2×2.
$\mathbf{M}$	Вещественная матрица Мюллера 4×4.
A	Площадь или полигон апертуры; смысл уточняется в конкретной формуле.
N_b	Число выходных пучков для одной ориентации.
N_{ori}	Число ориентаций в усреднении.

4. Координатные системы

4.1. Лабораторная система

Лабораторная система фиксирована относительно падающего света. Обычно падающий луч направлен вдоль одной из осей, а частица поворачивается матрицей Эйлера. Для любого вектора $\mathbf{x}$ в лабораторной системе используются скалярное произведение $\mathbf{a} \cdot \mathbf{b}$ и векторное произведение $\mathbf{a} \times \mathbf{b}$. Все направления лучей нормируются:

$$\|\mathbf{d}\| = 1, \qquad \|\mathbf{s}\| = 1, \qquad \|\mathbf{n}\| = 1.$$

4.2. Система частицы и повороты Эйлера

Пусть $\mathbf{x}_p$ — координаты вершины в системе частицы, а $\mathbf{x}_l$ — координаты в лабораторной системе. Поворот задается матрицей

$$\mathbf{x}_l = \mathbf{R}(\alpha, \beta, \gamma)\, \mathbf{x}_p.$$

Полная ориентация твёрдого тела задаётся тремя углами Эйлера или единичным кватернионом. В основной двухугловой квадратуре MBS-fast явно перебираются β и γ, а усреднение по лабораторному углу α представлено азимутальной сеткой направления рассеяния. Поэтому уменьшение числа азимутальных точек меняет не только вывод кривой, но и точность этого эквивалентного усреднения. Режим `--so3-quaternion` использует это тождество явно: квазислучайно выбирает только β, γ в фундаментальной области симметрии частицы, а интеграл по α вычисляет по азимуту рассеяния. Для независимой прямой проверки полной группы оставлен режим `--so3-full-quaternion`.

Удобная запись через элементарные повороты:

$$\mathbf{R}(\alpha, \beta, \gamma) = \mathbf{R}_z(\alpha)\, \mathbf{R}_y(\beta)\, \mathbf{R}_z(\gamma).$$

Здесь

$$\mathbf{R}_z(\chi) = \begin{pmatrix} \cos\chi & -\sin\chi & 0 \\ \sin\chi & \cos\chi & 0 \\ 0 & 0 & 1 \end{pmatrix}, \qquad \mathbf{R}_y(\chi) = \begin{pmatrix} \cos\chi & 0 & \sin\chi \\ 0 & 1 & 0 \\ -\sin\chi & 0 & \cos\chi \end{pmatrix}.$$

Таблица 6: Смысл углов Эйлера.

Символ	Смысл
α	Поворот вокруг лабораторной оси падающего луча или начальный азимут, если используется полная SO(3)-сетка.
β	Наклон оси частицы относительно падающего луча; мера содержит якобиан $\sin\beta$.
γ	Поворот частицы вокруг собственной оси после наклона. При $\beta = 0$ и $\beta = \pi$ все значения γ геометрически эквивалентны.

4.3. Система грани и апертуры

Для каждого выходного пучка строится локальная система апертуры. Нормаль апертуры $\mathbf{n}_b$ берется из последней грани или из нормали пучка, а две оси в плоскости обозначим $\mathbf{h}_b$ и $\mathbf{v}_b$:

$$\mathbf{h}_b \cdot \mathbf{n}_b = 0, \qquad \mathbf{v}_b \cdot \mathbf{n}_b = 0, \qquad \mathbf{h}_b \cdot \mathbf{v}_b = 0.$$

Точка $\mathbf{r}$ в лабораторной системе проектируется в координаты апертуры:

$$x = (\mathbf{r} - \mathbf{r}_c) \cdot \mathbf{h}_b, \qquad y = (\mathbf{r} - \mathbf{r}_c) \cdot \mathbf{v}_b,$$

где $\mathbf{r}_c$ — центр полигона пучка. Именно эти x, y попадают в `BeamEdgeData`: массивы вершин, наклоны ребер и признаки валидности ребер.

4.4. Система рассеяния

Для направления наблюдения $\mathbf{s}$ строятся две поперечные оси. В коде используется опорный вектор `vf` и второй вектор

$$\mathbf{v}_t = \frac{\mathbf{v}_f \times \mathbf{s}}{\|\mathbf{v}_f \times \mathbf{s}\|}.$$

Если направление близко к полюсу и знаменатель мал, используется предельная ветвь: при вырожденном азимуте базис рассеяния не определяется однозначно.

5. Трассировка лучей

5.1. Пересечение луча с гранью

Луч задается начальной точкой $\mathbf{r}_0$ и направлением $\mathbf{d}$:

$$\mathbf{r}(t) = \mathbf{r}_0 + t\mathbf{d}, \qquad t \geq 0.$$

Грань лежит в плоскости

$$(\mathbf{r} - \mathbf{p}_f) \cdot \mathbf{n}_f = 0,$$

где $\mathbf{p}_f$ — любая точка грани, $\mathbf{n}_f$ — нормаль грани. Формула пересечения:

$$t_f = \frac{(\mathbf{p}_f - \mathbf{r}_0) \cdot \mathbf{n}_f}{\mathbf{d} \cdot \mathbf{n}_f}.$$

Кандидат допустим, если $t_f > 0$, знаменатель не близок к нулю, а точка $\mathbf{r}(t_f)$ лежит внутри полигона грани. Для невыпуклой частицы этого недостаточно: найденная область может быть частично или полностью закрыта другой гранью.

5.2. Пучок как полигон

MBS-fast трассирует не одиночный луч, а полигональный пучок. Его апертура пересекается с полигоном грани. Если P_{in} — входной полигон, а F — грань в той же плоскости, то область попадания равна

$$P_{\text{hit}} = P_{\text{in}} \cap F.$$

Площадь $|P_{\text{hit}}|$ входит в критерии отсечения и нормировки. Ветвь с вырожденным полигоном удаляется.

5.3. Отражение и преломление

Пусть $\cos A = \mathbf{n} \cdot \mathbf{d}$. В `Splitting.cpp` используется вектор

$$\mathbf{r} = \frac{\mathbf{d}}{\cos A} - \mathbf{n},$$

после чего направление отраженной ветви записано как

$$\mathbf{d}_{\text{refl}} = \frac{\mathbf{r} - \mathbf{n}}{\|\mathbf{r} - \mathbf{n}\|}.$$

Такая запись эквивалентна обычной формуле зеркального отражения после учета знаковой конвенции нормали в коде. Для преломленной ветви код строит

$$s = \frac{1}{m_{\text{eff}}^2 \cos^2 A} - \|\mathbf{r}\|^2, \qquad \mathbf{d}_{\text{refr}} = \frac{\mathbf{r}/\sqrt{s} + \mathbf{n}}{\|\mathbf{r}/\sqrt{s} + \mathbf{n}\|}.$$

Если s становится неположительным, обычная преломленная ветвь отсутствует, и включается ветка полного внутреннего отражения.

5.4. Оптический путь

Фаза пучка включает оптический путь

$$\Phi_{\text{path}} = k\, L_{\text{opt}}.$$

Для внешней среды вклад длины равен геометрической длине. Для внутреннего участка длина умножается на эффективную вещественную часть показателя:

$$L_{\text{opt,in}} \approx \sqrt{\text{Re}(m^2)}\, L_{\text{geom}}.$$

В поглощающем варианте дополнительно используется мнимая часть показателя и интеграл с затуханием вдоль внутреннего пути.

5.5. Ограничение глубины

Параметр `--max-reflections N` ограничивает глубину внутренних отражений и преломлений. Если g — поколение дерева пучков, ветви с $g > N$ не трассируются. Без геометрического отсечения число ветвей могло бы расти как

$$N_{\text{beam}} \sim 2^g$$

но на практике рост ограничивают геометрия, поглощение и заданные пороги.

6. Коэффициенты Френеля

6.1. Вертикальная и горизонтальная поляризация

При каждом взаимодействии с гранью матрица Джонса умножается на два комплексных коэффициента: для локальной вертикальной и горизонтальной поляризаций. В коде они обозначены `cv/ch` или `Tv/Th`. Это диагональный оператор в локальном базисе Джонса:

$$\mathbf{J}_{\text{new}} = \begin{pmatrix} c_v & 0 \\ 0 & c_h \end{pmatrix} \mathbf{J}_{\text{old}} \quad \text{или} \quad \mathbf{J}_{\text{old}} \begin{pmatrix} c_v & 0 \\ 0 & c_h \end{pmatrix},$$

в зависимости от внутренней конвенции хранения строк и столбцов.

6.2. Обычное отражение и прохождение

В `ComputeRegularJonesParams` после вычисления $\cos G = \mathbf{n} \cdot \mathbf{d}_{\text{out}}$ используются величины

$$a_0 = m \cos A, \qquad a_1 = m \cos G,$$

$$D_v = a_1 + \cos A, \qquad D_h = a_0 + \cos G.$$

Для проходящей ветви:

$$c_v^{(T)} = \frac{2a_0}{D_v}, \qquad c_h^{(T)} = \frac{2a_0}{D_h}.$$

Для отраженной внутренней ветви:

$$c_v^{(R)} = \frac{\cos A - a_1}{D_v}, \qquad c_h^{(R)} = \frac{a_0 - \cos G}{D_h}.$$

Таблица 7: Обозначения в формулах обычной ветви Френеля.

Символ	Смысл
m	Комплексный показатель преломления частицы.
$\cos A$	Косинус угла между нормалью и падающим направлением в конвенции кода.
$\cos G$	Косинус угла между нормалью и выходящим направлением.

Символ	Смысл
D_v, D_h	Знаменатели Френеля для двух локальных поляризаций.
$c_v^{(T)}, c_h^{(T)}$	Множители Джонса для прошедшей ветви.
$c_v^{(R)}, c_h^{(R)}$	Множители Джонса для отражённой ветви.

6.3. Нормальное падение

При нормальном падении код использует упрощенные коэффициенты:

$$c_T = \frac{2m}{1+m}, \qquad c_R = \frac{1-m}{1+m}.$$

Для одной из поляризаций отраженная ветвь получает дополнительный знак, что связано с ориентацией локального поляризационного базиса после отражения:

$$\mathbf{J}_{\text{refl}} \leftarrow \begin{pmatrix} c_R & 0 \\ 0 & -c_R \end{pmatrix} \mathbf{J}.$$

6.4. Полное внутреннее отражение

При полном внутреннем отражении вычисляется

$$b = m_{\text{eff}}^2(1-\cos^2 A) - 1, \qquad q = \mathrm{i}\sqrt{\max(b,0)}.$$

Затем

$$c_v = \frac{\cos A - mq}{mq + \cos A}, \qquad c_h = \frac{m\cos A - q}{m\cos A + q}.$$

В непоглощающей среде их модуль близок к единице, но сохраняется фазовый сдвиг, определяющий поляризационные элементы матрицы Мюллера.

6.5. Старая конвенция знака Френеля

Параметр `--legacy-sign` включает прежнюю конвенцию знака в направлении вперёд. Она нужна только для воспроизведения старых результатов. В новых расчётах следует использовать поведение по умолчанию.

7. Матрицы Джонса и диады

7.1. Матрица пучка

Каждый пучок несет

$$\mathbf{J}_{\text{beam}} = \begin{pmatrix} J_{vv} & J_{vh} \\ J_{hv} & J_{hh} \end{pmatrix}.$$

Это амплитудная матрица: интенсивность определяется квадратами модулей, а интерференция сохраняется до суммирования всех матриц Джонса и последующего преобразования в матрицу Мюллера.

7.2. Фазовый множитель

Для внутреннего пучка функция `ComputeFnJones` использует фазу

$$\mathbf{F}_n(\mathbf{s}) = \mathbf{J}_{\text{beam}} \exp\left[\mathrm{i}k\left(L_{\text{proj}} - \mathbf{s}\cdot\mathbf{r}_c\right)\right].$$

Здесь L_{proj} хранится в поле `info.projLenght`, а $\mathbf{r}_c$ — центр пучка. Для внешнего пучка используется накопленный оптический путь:

$$\mathbf{F}_n = \mathbf{J}_{\text{beam}} \exp(\mathrm{i}kL_{\text{opt}}).$$

7.3. Поворот матрицы Джонса в базис рассеяния

В `HandlerPO::RotateJones` строятся векторы:

$$\mathbf{v}_t = \frac{\mathbf{v}_f \times \mathbf{s}}{\left\|\mathbf{v}_f \times \mathbf{s}\right\|},$$

$$\mathbf{N}_T = \mathbf{n}\times\mathbf{e}_T, \qquad \mathbf{N}_P = \mathbf{n}\times\mathbf{e}_P,$$

$$\mathbf{D}_T = \mathbf{d}_b\times\mathbf{e}_T, \qquad \mathbf{D}_P = \mathbf{d}_b\times\mathbf{e}_P.$$

Здесь $\mathbf{e}_T$ хранится в `info.beamBasis`, а $\mathbf{e}_P$ — в `beam.polarizationBasis`. Далее используется тождество тройного векторного произведения:

$$\mathbf{c}_T = \mathbf{s}\times\mathbf{N}_T + \mathbf{n}\times\mathbf{D}_T - \mathbf{s}\left[\mathbf{s}\cdot(\mathbf{n}\times\mathbf{D}_T)\right],$$

$$\mathbf{c}_P = \mathbf{s}\times\mathbf{N}_P + \mathbf{n}\times\mathbf{D}_P - \mathbf{s}\left[\mathbf{s}\cdot(\mathbf{n}\times\mathbf{D}_P)\right].$$

Матрица поворота:

$$\mathbf{R}_{\text{out}} = \frac{1}{2}\begin{pmatrix} \mathbf{c}_T\cdot\mathbf{v}_t & \mathbf{c}_P\cdot\mathbf{v}_t \\ \mathbf{c}_T\cdot\mathbf{v}_f & \mathbf{c}_P\cdot\mathbf{v}_f \end{pmatrix}.$$

В оптимизированных CPU- и CUDA-путях эта же матрица вычисляется без построения продольно спроецированных временных векторов. Поскольку $\mathbf{v}_f\cdot\mathbf{s} = \mathbf{v}_t\cdot\mathbf{s} = 0$, при обозначениях $\mathbf{X}_T = \mathbf{n}\times\mathbf{D}_T$ и $\mathbf{X}_P = \mathbf{n}\times\mathbf{D}_P$ справедливо точное равенство

$$\mathbf{R}_{\text{out}} = \frac{1}{2}\begin{pmatrix} \mathbf{X}_T\cdot\mathbf{v}_t - \mathbf{N}_T\cdot\mathbf{v}_f & \mathbf{X}_P\cdot\mathbf{v}_t - \mathbf{N}_P\cdot\mathbf{v}_f \\ \mathbf{X}_T\cdot\mathbf{v}_f + \mathbf{N}_T\cdot\mathbf{v}_t & \mathbf{X}_P\cdot\mathbf{v}_f + \mathbf{N}_P\cdot\mathbf{v}_t \end{pmatrix}.$$

Отброшенный член параллелен $\mathbf{s}$ и обнуляется при проекции на оба вектора выходного базиса. Это тождественное преобразование, а не приближение поляризации; оно исключает две временные векторные величины и их продольные проекции для каждого вклада пучка в направление наблюдения.

7.4. Полный вклад пучка в матрицу Джонса

В `ApplyDiffractionFast` итоговый вклад одного пучка:

$$\mathbf{J}_b(\theta,\phi) = F_{\mathrm{edge},b}(\theta,\phi)\,\mathbf{R}_{\mathrm{out},b}(\theta,\phi)\,\mathbf{F}_{n,b}(\theta,\phi).$$

Таблица 8: Множители полного вклада Джонса.

Множитель	Смысл
$F_{\mathrm{edge},b}$	Скалярный интеграл физической оптики по полигональной апертуре пучка b.
$\mathbf{R}_{\mathrm{out},b}$	Переход из локального базиса Джонса пучка в базис рассеяния.
$\mathbf{F}_{n,b}$	Накопленная матрица Джонса с фазой оптического пути и поправкой на направление наблюдения.

7.5. Экспериментальная ветвь Карчевского

Параметр `--karczewski` заменяет `RotateJones` на `KarczewskiJones`. Эта ветвь экспериментальна. Она сохраняет M_{11}, поскольку не меняет норму Фробениуса оператора Джонса:

$$M_{11} = \frac{1}{2}\,\|\mathbf{J}\|_F^2 = \frac{1}{2}\sum_{i=1}^{2}\sum_{j=1}^{2}|J_{ij}|^2.$$

Поляризационные элементы M_{33}, M_{34} и M_{44} могут отличаться, поскольку меняется структура матрицы Джонса в базисе рассеяния.

Поперечные нормы вычисляются через codehypot; каждый базис проверяется до нормировки. При вырожденной полюсной конфигурации или не конечном результате программа возвращается к основному codeRotateJones. Эта защита исключает codeNaN, но не превращает неполную реализацию конвенции GOAD в физический эталон.

8. Переход к матрице Мюллера

8.1. Полная формула из кода

Пусть

$$\mathbf{J} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}.$$

В `src/math/Mueller.cpp` матрица Мюллера строится из билинейных комбинаций в следующей конвенции:

$$M_{00} = \frac{|a|^2+|b|^2+|c|^2+|d|^2}{2}, \quad M_{01} = \frac{|a|^2-|b|^2+|c|^2-|d|^2}{2},$$
$$M_{10} = \frac{|a|^2+|b|^2-|c|^2-|d|^2}{2}, \quad M_{11} = \frac{|a|^2-|b|^2-|c|^2+|d|^2}{2}.$$

Далее:

$$
\begin{aligned}
M_{02} &= -\operatorname{Re}(a\overline{b}) - \operatorname{Re}(d\overline{c}), & M_{03} &= \operatorname{Im}(d\overline{c}) - \operatorname{Im}(a\overline{b}),\\
M_{12} &= \operatorname{Re}(d\overline{c}) - \operatorname{Re}(a\overline{b}), & M_{13} &= -\operatorname{Im}(a\overline{b}) - \operatorname{Im}(d\overline{c}),\\
M_{20} &= -\operatorname{Re}(a\overline{c}) - \operatorname{Re}(d\overline{b}), & M_{21} &= \operatorname{Re}(d\overline{b}) - \operatorname{Re}(a\overline{c}),\\
M_{30} &= \operatorname{Im}(a\overline{c}) - \operatorname{Im}(d\overline{b}), & M_{31} &= \operatorname{Im}(d\overline{b}) + \operatorname{Im}(a\overline{c}),\\
M_{22} &= \operatorname{Re}(a\overline{d}) + \operatorname{Re}(b\overline{c}), & M_{23} &= \operatorname{Im}(a\overline{d}) - \operatorname{Im}(b\overline{c}),\\
M_{32} &= -\operatorname{Im}(a\overline{d}) - \operatorname{Im}(b\overline{c}), & M_{33} &= \operatorname{Re}(a\overline{d}) - \operatorname{Re}(b\overline{c}).
\end{aligned}
$$

Таблица 9: Обозначения при переходе от матрицы Джонса к матрице Мюллера.

Символ	Смысл
a, b, c, d	Комплексные элементы итоговой матрицы Джонса для одного направления наблюдения.
$\overline{z}$	Комплексное сопряжение.
$\lvert z\rvert^2 = z\overline{z}$	Квадрат модуля комплексной амплитуды.
$\operatorname{Re}(z), \operatorname{Im}(z)$	Вещественная и мнимая части.
M_{ij}	Элементы Мюллера в нулевой индексации кода; в файлах записываются как $M11, M12, \ldots, M44$.

8.2. Когерентная и некогерентная сумма

По умолчанию для одной ориентации:

$$\mathbf{J}_{\text{tot}}(\theta,\phi) = \sum_{b=1}^{N_b} \mathbf{J}_b(\theta,\phi), \qquad \mathbf{M}(\theta,\phi) = \mathcal{M}(\mathbf{J}_{\text{tot}}).$$

С параметром `--incoherent`:

$$\mathbf{M}_{\text{incoh}}(\theta,\phi) = \sum_{b=1}^{N_b} \mathcal{M}(\mathbf{J}_b(\theta,\phi)).$$

Разница принципиальная: в первом случае сохраняются интерференционные члены между разными пучками, во втором они исчезают.

Когерентная сумма выполняется только между лучевыми путями одной частицы при одной фиксированной ориентации. Для ансамбля случайно и независимо ориентированных частиц программа сначала строит $\mathcal{M}(\mathbf{J}_{\text{tot}})$ для каждой ориентации, а затем усредняет матрицы Мюллера с ориентационными весами. Складывать матрицы Джонса разных ориентаций нельзя без координат частиц и относительных фаз падающего поля. Поэтому устаревший параметр `--coherent-orientations` отключен: его прежняя реализация фактически выполняла обычное некогерентное усреднение ориентаций.

Одночастичная модель не воспроизводит когерентное обратное рассеяние среды, возникающее из интерференции взаимно обратных многократных путей между разными частицами. Для такой постановки нужен многочастичный решатель с координатами частиц и межчастичным распространением поля.

Отсечения `--beam-cutoff*` и `--trace-cutoff*` удаляют амплитуды до когерентной суммы и могут исказить перекрестные члены даже при малой удаленной интенсивности. Обратное направление следует проверять повторным расчетом с `--cutoff-profile safe` или `--cutoff-profile off`.

8.3. Повороты матрицы Мюллера

Поворот базиса Стокса на угол χ действует только на компоненты Q, U:

$$\mathbf{R}_M(\chi) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\chi & -\sin\chi & 0 \\ 0 & \sin\chi & \cos\chi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

`RightRotateMueller` умножает матрицу справа и меняет входной базис Стокса:

$$\mathbf{M}' = \mathbf{M}\,\mathbf{R}_M(\chi_{\text{in}}).$$

`LeftRotateMueller` умножает слева, то есть меняет выходной базис:

$$\mathbf{M}' = \mathbf{R}_M(\chi_{\text{out}})\,\mathbf{M}.$$

`RotateMueller` делает обе операции в одной функции:

$$\mathbf{M}' = \mathbf{R}_M(\chi_{\text{out}})\,\mathbf{M}\,\mathbf{R}_M(\chi_{\text{in}}).$$

9. Дифракция в приближении физической оптики

9.1. Интеграл по апертуре

Для выходного пучка с апертурой A_b скалярная часть PO-дальнего поля записывается как

$$F_b(\mathbf{q}) = \int_{A_b} \exp(\mathrm{i}k\,\mathbf{q}\cdot\mathbf{r})\ \mathrm{d}A, \qquad \mathbf{q} = \mathbf{d}_b - \mathbf{s}.$$

9.2. Переход к ребрам полигона

В системе апертуры:

$$A_q = \mathbf{q}\cdot\mathbf{h}_b, \qquad B_q = \mathbf{q}\cdot\mathbf{v}_b.$$

Для вершины j:

$$\Phi_j = k(A_q x_j + B_q y_j).$$

Если ребро описано через $y = m_j x + c_j$, то один из вариантов суммы по рёбрам имеет вид

$$S_x = \frac{1}{B_q}\sum_j \frac{\exp(\mathrm{i}\Phi_{j+1}) - \exp(\mathrm{i}\Phi_j)}{A_q + m_j B_q}.$$

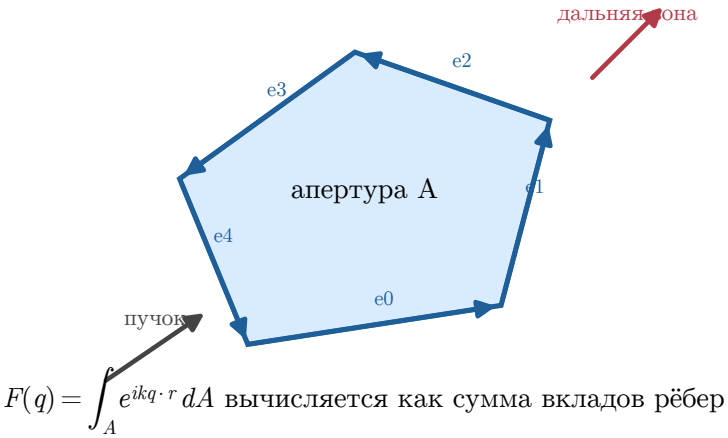


Рис. 2: Интеграл по апертуре сводится к сумме вкладов рёбер полигона. Такой способ быстрее и точнее прямого численного интегрирования по площади.

Если удобнее параметризовать ребро как $x = \mu_j y + \eta_j$, используется симметричная форма

$$S_y = -\frac{1}{A_q} \sum_j \frac{\exp(\mathrm{i}\Phi_{j+1}) - \exp(\mathrm{i}\Phi_j)}{A_q \mu_j + B_q}.$$

В коде выбор между ветками зависит от того, какой знаменатель устойчивее. Для малых A_q и B_q используется предельная формула площади:

$$F_b(\mathbf{0}) \approx C_k \, |A_b|,$$

где C_k — комплексный нормировочный множитель, соответствующий выбранной форме интеграла в коде.

Таблица 10: Обозначения в интеграле по рёбрам.

Символ	Смысл
$\mathbf{q}$	Разность направления пучка и направления наблюдения.
A_q, B_q	Компоненты $\mathbf{q}$ в плоскости апертуры.
(x_j, y_j)	Вершина полигона пучка в системе апертуры.
m_j, μ_j	Наклоны ребра: dy/dx и dx/dy.
Φ_j	Фаза комплексной экспоненты в вершине j.
S_x, S_y	Две эквивалентные формы суммы по рёбрам, выбираемые по устойчивости.

9.3. Почему важны точки 0 и 180 градусов

В направлениях точно вперёд и точно назад часть базисов и знаменателей становится вырожденной или почти вырожденной:

$$\|\mathbf{v}_f \times \mathbf{s}\| \to 0, \qquad A_q \to 0, \quad B_q \to 0.$$

Поэтому в этих точках особенно важны:

- одинаковая обработка концов углового диапазона на CPU и GPU;
- одинаковая формула предела интеграла по рёбрам;
- одинаковый вес γ на полюсах по β при `--pole`;
- отсутствие расхождения в знаке коэффициентов Френеля и в соглашении о фазе;
- одинаковая точность: одинарная точность может заметно отличаться от двойной именно в почти вырожденных ветвях.

9.4. Поглощение

Если $\mathrm{Im}(m) > 0$ или указан `--abs`, внутренняя часть пучка получает затухание:

$$\exp(\mathrm{i}kmL) = \exp(\mathrm{i}k\,\mathrm{Re}(m)L)\,\exp(-k\,\mathrm{Im}(m)L).$$

Тогда вклад длинных внутренних путей уменьшается. В численном смысле это помогает сходимости дерева лучей, но меняет физическую интенсивность и поляризационные элементы.

10. Обрезка пучков в невыпуклых частицах и агрегатах

10.1. Затенение апертуры другими гранями

Для выпуклой частицы пересечение пучка с ближайшей гранью обычно однозначно. У невыпуклой частицы другая грань может закрывать часть апертуры. Тогда из полигона текущего пучка вычитают проекцию блокирующей грани:

$$P_{\mathrm{vis}} = P_{\mathrm{subj}} \setminus \bigcup_{m=1}^{N_c} \Pi_{\mathrm{subj}}(C_m).$$

Таблица 11: Объекты геометрической обрезки.

Символ	Смысл
P_{subj}	Полигон текущего пучка, который проверяется на видимость.
C_m	Блокирующая грань или полигон, потенциально закрывающие текущий пучок.
Π_{subj}	Проекция блокирующего объекта на плоскость текущего пучка вдоль направления распространения.

Символ	Смысл
P_{vis}	Видимая часть полигона после вычитания проекций.
N_c	Число кандидатов после быстрых фильтров.

10.2. Агрегаты

В агрегате блокирующая грань может принадлежать другой компоненте:

$$C_m \in \bigcup_{p=1}^{N_{\text{part}}} \mathcal{F}_p,$$

где $\mathcal{F}_p$ — множество граней компоненты p.

10.3. Предварительный отбор по проекциям AABB

Точное вычитание полигонов требует значительных затрат. Поэтому `--trace-prefilter` сначала строит ограничивающий прямоугольник проекции блокирующей грани:

$$B(C_m) = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}].$$

Если

$$B(C_m) \cap B(P_{\text{subj}}) = \emptyset,$$

то точная обрезка не нужна. Это не меняет физику, а только уменьшает число дорогих геометрических операций.

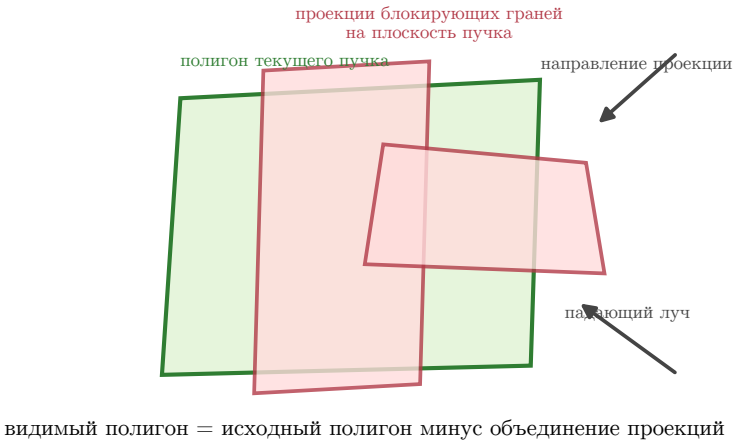


Рис. 3: Обрезка невыпуклого пучка: проекция блокирующей грани удаляет закрытую часть его полигона.

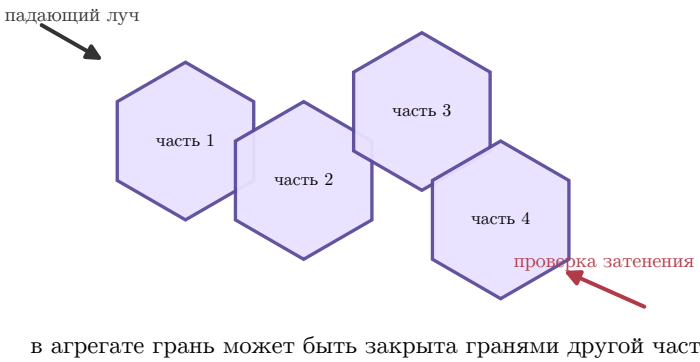


Рис. 4: Для агрегата затенение идет не только внутри одной компоненты, но и между разными компонентами.

11. Размер частицы и масштабирование геометрии

В этом разделе различаются геометрические размеры, задаваемые генератору, и производные характеристики, используемые для сопоставления частиц. Масштаб всегда применяется к фактически построенной или прочитанной геометрии.

11.1. Встроенные частицы и геометрия из файла

Частица задаётся либо параметром `--particle`, после которого следуют тип и его параметры, либо параметром `--particle-file` с именем файла. В первом случае геометрию создаёт встроенный генератор, во втором она читается из файла. Одновременное задание двух источников запрещено. Параметр `--save-geometry FILE` сохраняет фактически использованную геометрию после масштабирования и определения её выпуклости; затем расчёт продолжается.

Координаты файловой частицы и вся внутренняя геометрия трассировки хранятся в двойной точности. Загрузчик сдвигает координаты относительно не зависящего от порядка центра ограничивающего параллелепипеда и удаляет только относительный к масштабу шум текстовой сериализации. Глобальный перенос меняет лишь общую оптическую фазу; каноническая форма сохраняет матрицу Мюллера, малые рёбра и порядок граней при больших абсолютных координатах.

Таблица 12: Основные встроенные частицы.

Тип	Форма	Параметры
1	Гексагональный столбик или пластинка	`L D`: осевая длина и диаметр между противоположными вершинами. Отношение L/D определяет вытянутую или сплюснутую форму.
2	Пуля	`L D`: полная осевая длина и диаметр; высота пирамидальной головки вычисляется из фиксированного угла 62°.
3	Розетка из пуль	`L D [CAP]`: шесть одинаковых ветвей; без `CAP` используется та же конструкция 62°.
4	Дрокстал	`SCALE`: масштаб фиксированного шаблона; старая форма `L D SCALE` принимает, но игнорирует `L,D`.
10	Полый гексагональный столбик	`L D CAVITY\_DEG`: верхняя и нижняя пирамидальные полости; угол должен быть положительным и меньше $\arctan(L/D)$.
12	Двухкомпонентный гексагональный агрегат	`L D 2`: ровно две одинаковые восьмигранные колонки в фиксированном взаимном положении.

Тип	Форма	Параметры
999	Встроенный контрольный агрегат	`SCALE`: масштаб фиксированной регрессионной геометрии.

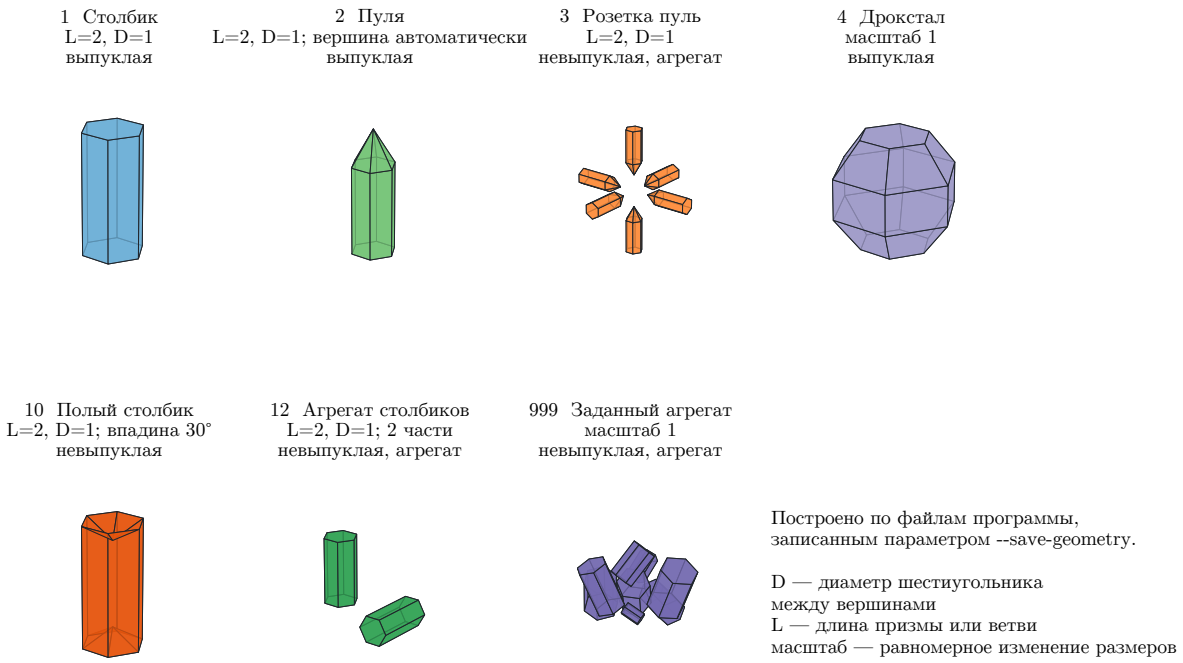


Рис. 5: Встроенные формы. Изображения построены по файлам из каталога `examples/particles`, записанным генераторами самой программы.

11.2. Внутренний формат частицы и экспорт геометрии

Первые три непустые строки имеют строгий смысл:

```
CONCAVE
AGGREGATE [FACETS_PER_PART]
BETA_DOMAIN_DEG GAMMA_DOMAIN_DEG

x0 y0 z0
x1 y1 z1
x2 y2 z2

x0 y0 z0      # следующая грань
...
```

`CONCAVE` равен 0 или 1. `AGGREGATE` равен 0 для одной частицы либо записывается как `1` `FACETS_PER_PART` для компонентов с одинаковым числом граней. Третья строка задаёт ширину

фундаментальной области по β и γ в градусах. Пустая строка разделяет грани; у каждой грани должно быть не меньше трёх конечных неколлинеарных вершин. Символ `##` начинает комментарий. Порядок вершин определяет направление нормали, поэтому обход всех граней должен быть согласован и задавать внешние нормали. Каждая грань должна быть простым плоским выпуклым многоугольником, а каждая компонента частицы — замкнутой многообразной поверхностью. Допускается не более 64 вершин в грани и 256 граней в частице; эти условия проверяются до трассировки.

Экспорт записывает координаты с 17 значащими цифрами и проверяет метаданные агрегата. Файл содержит уже масштабированную рабочую геометрию, поэтому подходит для точного повторения расчёта, визуализации обрезки пучков и проверки цикла «запись–чтение»:

```
gpu/bin/mbs_po_gpu_double --method po \
  --particle 10 20 12 15 \
  --save-geometry concave.particle ...
gpu/bin/mbs_po_gpu_double --method po \
  --particle-file concave.particle ...
```

Экспорт состоит из `FILE` и `FILE.visibility`. Боковой файл хранит два флага видимости каждой грани и автоматически читается вместе с геометрией. Для точного повторения расчёта вогнутой встроенной частицы нужны оба файла. Рабочая геометрия не записывается автоматически; её следует явно сохранить параметром `--save-geometry FILE` (алиас `--save-particle`).

11.3. Эквивалентный радиус и k_{eq}

Эквивалентный радиус по объему:

$$r_{\mathrm{eq}} = \left(\frac{3V}{4\pi}\right)^{1/3}.$$

Эквивалентный размерный параметр:

$$k_{\mathrm{eq}} = \frac{2\pi r_{\mathrm{eq}}}{\lambda}.$$

Если частица масштабируется параметром `--k-eq X`, то масштаб s выбирается из

$$s = \frac{k_{\mathrm{eq,target}}\lambda}{2\pi r_{\mathrm{eq,old}}}.$$

Таблица 13: Флаги размера.

Флаг	Смысл
`--resize-dmax-um SIZE`	Масштабировать геометрию из файла так, чтобы $D_{\max} =$ `SIZE`.
`--k-eq X`	Масштабировать так, чтобы $k_{\mathrm{eq}} = X$. Требуется `--wavelength-um`.
`--dmax-grid A B N`	Логарифмический диапазон $D_{\max}$.
`--k-eq-grid A B N`	Логарифмический диапазон k_{eq}.

Флаг	Смысл
`--k-eq-list FILE`	Точный список положительных k_{eq} из файла.

Часть III

Ориентации и угловые сетки

12. Ориентационное усреднение

12.1. Интеграл по ориентациям

Для случайно ориентированных частиц полная средняя по группе вращений имеет меру

$$\langle \mathbf{M} \rangle = \frac{1}{8\pi^2} \int_0^{2\pi} \int_0^{\pi} \int_0^{2\pi} \mathbf{M}(\alpha, \beta, \gamma) \sin\beta \, \mathrm{d}\alpha \, \mathrm{d}\beta \, \mathrm{d}\gamma.$$

В основной двухугловой схеме MBS-fast интеграл по лабораторному углу α реализован через азимутальную сетку рассеяния. После этого явно усредняются β и γ:

$$\overline{\mathbf{M}}(\theta) = \frac{1}{W} \sum_{i=1}^{N_{\mathrm{ori}}} w_i \, \mathbf{M}_i(\theta), \qquad W = \sum_{i=1}^{N_{\mathrm{ori}}} w_i.$$

В непрерывной форме редуцированная двухугловая часть равна

$$\overline{\mathbf{M}}(\theta) = \frac{1}{4\pi} \int_0^{2\pi} \int_0^{\pi} \mathbf{M}(\theta; \beta, \gamma) \sin\beta \, \mathrm{d}\beta \, \mathrm{d}\gamma.$$

Удобная переменная $\mu = \cos\beta$ дает

$$\sin\beta \, \mathrm{d}\beta = -\,\mathrm{d}\mu,$$

поэтому квадратура по μ естественным образом учитывает сферическую меру. Эта формула применима после усреднения по α; сама по себе сетка $\beta \times \gamma$ не заменяет произвольную трёхугловую квадратуру.

12.2. Регулярная сетка по β, γ

Если ячейка по β имеет границы $\beta_{j-1/2}$ и $\beta_{j+1/2}$, её вес равен

$$w_\beta(j) = \cos\beta_{j-1/2} - \cos\beta_{j+1/2}.$$

Для равномерной сетки по γ

$$w_\gamma = \frac{\gamma_{\max} - \gamma_{\min}}{N_\gamma}.$$

Итоговый вес:

$$w_{j,l} = w_\beta(j) w_\gamma(l).$$

12.3. Флаг `--pole`

На полюсах $\beta = 0$ и $\beta = \pi$ все γ описывают одну и ту же физическую ориентацию:

$$\mathbf{R}_z(\gamma)\mathbf{R}_y(0) = \mathbf{R}_z(\gamma),$$

Поэтому `--pole` оставляет на полюсе одну точку по γ, но назначает ей вес всего интервала:

$$w_{\text{pole}} = w_\beta \, (\gamma_{\max} - \gamma_{\min}).$$

Эту специальную ветвь необходимо одинаково реализовывать на CPU и GPU. Расхождение полюсного веса или выбора базиса прежде всего проявляется в направлениях 0° и 180°.

12.4. Сетка по дифракционной оценке

Канонический параметр `--diffraction-sampling Q` строит сетки ориентаций и направлений рассеяния из одной оценки дифракционного углового масштаба:

$$\xi = 0.69 \, \frac{\lambda}{l_{\max}}.$$

Здесь ξ выражена в радианах, а $l_{\max}$ — максимальная длина ребра среди всех граней уже масштабированной частицы:

$$l_{\max} = \max_f \max_i \left\| \mathbf{r}_{f,i+1} - \mathbf{r}_{f,i} \right\|, \qquad \mathbf{r}_{f,n_f} = \mathbf{r}_{f,0}.$$

Это не диаметр частицы и не максимальное расстояние между произвольными вершинами. Параметр Q задаёт число угловых интервалов на масштабе ξ:

$$\Delta_{\text{target}} = \frac{\xi}{Q}.$$

Следовательно, $Q = 1$ соответствует шагу ξ, $Q = 2$ — шагу $\xi/2$, а $Q = 4$ — шагу $\xi/4$. Чем больше Q, тем мельче сетка и дороже расчёт. Число Q не является общим числом ориентаций или числом точек на кольце.

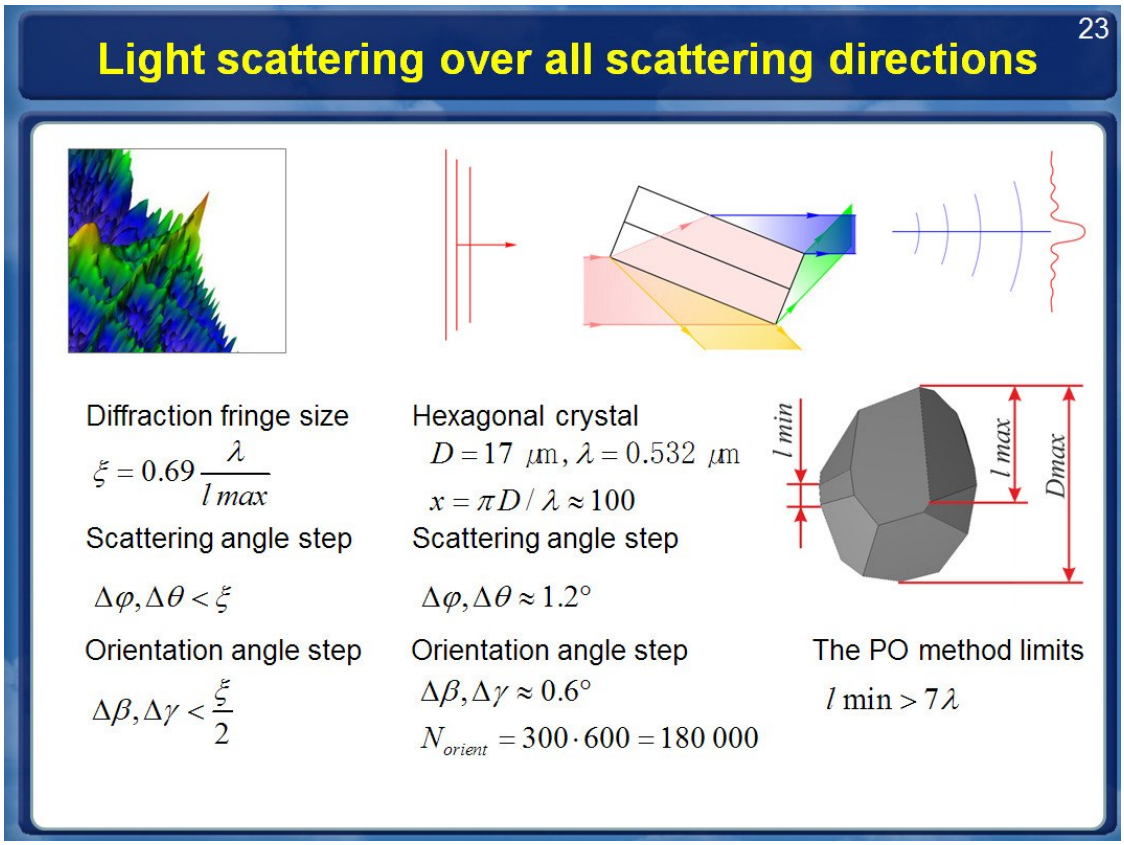


Рис. 6: Исходная оценка углового разрешения по дифракционной полосе. Показанное условие $\Delta\phi, \Delta\theta < \xi$ соответствует $Q_{\text{scat}} \geq 1$, а $\Delta\beta, \Delta\gamma < \xi/2$ — $Q_{\text{ori}} \geq 2$. В программе $l_{\max}$ вычисляется как максимальная длина ребра среди всех граней частицы, отдельно от полного диаметра $D_{\max}$.

Для ориентаций программа вычисляет

$$N_\beta = \max\left(3, \left\lceil \frac{\beta_{\max} - \beta_{\min}}{\xi/Q_{\text{ori}}} \right\rceil\right), \qquad N_\gamma = \max\left(3, \left\lceil \frac{\gamma_{\max} - \gamma_{\min}}{\xi/Q_{\text{ori}}} \right\rceil\right).$$

Фундаментальные диапазоны β и γ берутся из симметрии частицы; `--mirror-gamma` дополнительно сокращает область по γ. Из-за округления фактические шаги могут быть немного меньше целевого ξ/Q.

Общий флаг назначает $Q_{\text{ori}} = Q_{\text{scat}} = Q$. Если требуется другая плотность только для ориентаций, применяется переопределение `--orientation-diffraction-sampling Q`. Например,

```
--diffraction-sampling 2 \
--orientation-diffraction-sampling 4
```

задаёт $\xi/4$ для β, γ и сохраняет $\xi/2$ для θ, ϕ. Параметр `--orientation-diffraction-sampling` можно использовать без общего флага и совместить с явно заданным `--scattering-grid`.

Исторические параметры `--diffraction-grid`, `--diffraction-limit-grid` и `--ring-points` сохранены только для совместимости старых команд и показаны в `--help-debug`. Их нельзя смешивать с новой схемой: прежний алгоритм использовал другую зависимость числа ориентаций

от делителя и скрытое значение `ring-points=3`. Расчёты, выполненные до исправления определения $l_{\max}$, могли строить сетку по диаметру частицы; повторный запуск такой команды в новой версии правильно использует максимальное ребро и поэтому может выбрать другое число ориентаций.

12.5. Квазислучайные и случайные выборки

Квазислучайные режимы заменяют регулярную сетку набором точек низкой дискрепантности. Идея:

$$\overline{\mathbf{M}}_N = \frac{1}{N}\sum_{i=1}^{N} \mathbf{M}(T(\mathbf{u}_i)),$$

где $\mathbf{u}_i \in [0,1)^d$ — Sobol/Hammersley/lattice point, а T переводит единичный куб в углы ориентации с правильной мерой.

Таблица 14: Ориентационные методы.

Флаг	Метод
`--fixed-orientation B G`	Одна ориентация β, γ в градусах; удобна для отладки CPU/GPU.
`--orientation-file FILE`	Пары β, γ в градусах, по одной паре на строку.
`--euler-grid NB NG`	Явная регулярная сетка $\beta \times \gamma$.
`--diffraction-sampling Q`	Регулярная сетка ориентаций и автоматическая сетка рассеяния с шагом ξ/Q.
`--orientation-diffraction-sampling Q`	Только регулярная сетка ориентаций с шагом ξ/Q.
`--sobol N`	Квазислучайная последовательность Соболя.
`--sobol-seed N SEED`	Последовательность Соболя с заданным перемешиванием Оуэна.
`--so3-quaternion N`	Хаммерсли в области симметрии; лабораторный α интегрируется по азимуту рассеяния.
`--so3-mirror-audit`	При чётном N явно трассировать вложенные отражённые пары для проверки `--mirror-gamma`.
`--so3-full-quaternion N`	Прямая выборка полной группы $SO(3)$ через кватернионы Шумейка для независимой проверки.
`--hammersley N`	Выборка Хаммерсли.
`--lattice N`	Ранговая решётка первого порядка.
`--monte-carlo N`	Псевдослучайные ориентации; ошибка обычно убывает как $N^{-1/2}$.

12.6. Квадратура SO(3) с учетом симметрии

Для изотропного ансамбля мера Хаара в принятом соглашении углов равна

$$dR \propto \sin\beta \, d\beta \, d\gamma \, d\alpha.$$

Режим `--so3-quaternion N` использует фундаментальную область частицы $0 \le \beta \le \beta_{\mathrm{sym}}$, $0 \le \gamma < \gamma_{\mathrm{sym}}$. Для $i = 0, \dots, N-1$ строятся узлы

$$u_i = \frac{i + 1/2}{N}, \qquad v_i = \mathrm{radical_inverse}_2(i),$$

$$\beta_i = \arccos\bigl[1 - (1 - \cos\beta_{\mathrm{sym}})u_i\bigr], \qquad \gamma_i = \gamma_{\mathrm{sym}} v_i, \qquad w_i = \frac{1}{N}.$$

Равномерно распределён $\cos\beta$, а не сам β; нормированные веса удовлетворяют $\sum_i w_i = 1$. Флаг `--sym Sb Sg` переопределяет область: $\beta_{\mathrm{sym}} = \pi/S_b$, $\gamma_{\mathrm{sym}} = 2\pi/S_g$. Его допустимо использовать только при реальной симметрии геометрии.

Для парной проверки зеркальной реконструкции к чётному полному числу узлов добавляется `--so3-mirror-audit`. При $K = N/2$ те же формулы применяются для $i = 0, \dots, K-1$ с $u_i = (i+1/2)/K$ и $\gamma_i = (\gamma_{\mathrm{sym}}/2)v_i$, после чего явно трассируются пары (β_i, γ_i) и $(\beta_i, \gamma_{\mathrm{sym}} - \gamma_i)$ с весом $1/N$. Эта сетка точно вложена в расчёт из K узлов с `--mirror-gamma`. Без контрольного флага обычная полнодоменная последовательность Хаммерсли не меняется.

С флагом `--mirror-gamma` вычисляется только диапазон $0 \le \gamma < \gamma_{\mathrm{sym}}/2$. Вторая половина восстанавливается до квадратуры по α:

$$\mathbf{M}_{\mathrm{full}}(\theta, \phi) = \frac{1}{2}\left[\mathbf{M}_{\mathrm{half}}(\theta, \phi) + \mathbf{P}\mathbf{M}_{\mathrm{half}}(\theta, -\phi)\mathbf{P}\right], \qquad \mathbf{P} = \mathrm{diag}(1, 1, -1, -1).$$

Преобразование допустимо только при наличии у всей оптической частицы соответствующей плоскости отражения, включая видимость граней и назначенные материалы, а также при численной инвариантности выбранного трассировщика ФО относительно такого отражения. По одному файлу частицы программа не может доказать это свойство; для каждого нового класса частиц и набора параметров нужен контрольный расчёт с полным диапазоном γ. N остаётся числом реально трассируемых ориентаций β, γ. Для строгой парной проверки сравниваются `--so3-quaternion 4096 --so3-mirror-audit` и `--so3-quaternion 2048 --mirror-gamma`. Их базовые узлы совпадают, поэтому разность измеряет численную ошибку зеркальной реконструкции, а не шум двух разных выборок Хаммерсли. Обычную полнодоменную сетку также надо сравнить с независимым плотным результатом, а обратное рассеяние проверить отдельно.

Угол α отдельно не трассируется. Поворот вокруг падающего луча эквивалентен изменению азимута рассеяния ϕ, поэтому каждая строка θ накапливается как

$$\overline{M}(\theta) = \frac{1}{N_\phi} \sum_{\phi} M(\theta, \phi) L(-\phi).$$

Матрица L поворачивает базис Стокса; её блок Q/U содержит $\cos(2\phi)$ и $\sin(2\phi)$. Умножение выполняется справа, поскольку меняется базис падающей поляризации. В направлениях $\theta = 0$

и $\theta = \pi$ азимутальный базис вырожден: код усредняет без L, затем накладывает точную форму матрицы для прямого или обратного направления.

Режим требует $N_\phi \geq 2$. Для итогового запуска его следует сочетать с `--scattering-diffraction-sampling Q` и `--latitude-phi-grid`. Более медленный `--so3-full-quaternion N` нужен только для проверки: там N означает число полных трёхмерных поворотов, а в быстром режиме N — число действительно трассируемых ориентаций β, γ; квадратуру по α выполняет сетка ϕ.

12.7. Адаптивный выбор числа ориентаций

При сравнении кандидатов по N_ϕ, глубине отражений и сетке Эйлера для двух приближений A и B ошибка одной строки по θ равна

$$e_{\mathrm{row}} = \max\left(\frac{|M_{11}^A - M_{11}^B|}{\max(|M_{11}^A|, |M_{11}^B|, \varepsilon_0)}, \max_{(i,j)\neq(1,1)} \frac{|R_{ij}^A - R_{ij}^B|}{\max(1, |R_{ij}^A|, |R_{ij}^B|)} \right), \qquad R_{ij} = \frac{M_{ij}}{M_{11}}.$$

Здесь $R_{ij} = M_{ij}/M_{11}$, а ε_0 защищает деление около нулей. Затем берётся максимум по строкам θ и добавляется относительное изменение интеграла $\int M_{11}\, \mathrm{d}\Omega$. Для глубины отражений также проверяется выходная энергия. При подборе сетки θ значения в точках $1/3$ и $2/3$ каждого интервала сравниваются с интерполяцией, после чего отдельно проверяется интеграл M_{11}. При подборе числа точек Соболя проверяются входная энергия и все 16 элементов на контрольных углах.

По умолчанию для числа ориентаций, N_ϕ, глубины отражений и сетки Эйлера требуются два последовательных успешных уточнения; сетка θ уточняет интервалы напрямую. Параметр `--convergence-passes N` меняет число таких подтверждений. В совместных адаптивных режимах достижение верхнего предела считается ошибкой. Отдельный режим `--adaptive-orientations` выдаёт предупреждение и сохраняет лучший результат, но не подтверждает сходимость.

12.8. Файл настроек адаптивной сходимости

Параметр `--adaptive-config FILE` загружает все пределы рабочего расчёта из одного текстового файла. Полный шаблон с пояснением каждой строки находится в файле `configs/adaptive.example.conf`. Формат строгий: `key = value`; после символа `#` или `;` разрешен комментарий. Например:

```
reflections.min = 2
reflections.max = 30
reflections.tolerance = 0.01  # 1 percent
alpha.min_points = 12
alpha.max_points = 2400
alpha.tolerance = 0.01
orientations.min = 128
orientations.max = 262144
orientations.tolerance = 0.02
```

Группы `reflections`, `alpha`, `theta`, `orientations`, `beta` и `gamma` задают минимум, максимум и свою относительную точность. Группа `controller` задает число последовательных успешных

уточнений, диапазон pilot-набора и максимальное число совместных проходов. Значение `0.01` означает относительную погрешность 1%, а не 0.01%.

Если индивидуальная точность отсутствует в файле, используется общий `EPS` из `--auto`, `--autofull` или другого выбранного режима. Индивидуальная точность из файла имеет приоритет над общим `EPS`. Явные флаги командной строки `--max-*`, `--max-reflections` и `--convergence-passes` имеют приоритет над файлом. Неизвестный или повторный ключ, неверный диапазон и неправильный тип значения останавливают запуск до расчета с указанием файла, номера строки и способа исправления.

```
cpu/bin/mbs_po_mpi --method po --particle 1 100 70 \
  --refractive-index 1.3116 0 --wavelength-um 0.532 \
  --autofull 0.02 \
  --adaptive-config configs/adaptive.example.conf \
  --backend cpu --threads 64 --output converged --close
```

12.9. Совместный адаптивный подбор параметров

Параметры не являются независимыми. Глубина n меняет дерево лучей; N_ϕ меняет эквивалентное усреднение по α; сетка θ определяет, на каких углах контролируется результат; число ориентаций меняет каждую усреднённую строку. Поэтому общий порядок имеет вид

$$n \longrightarrow N_\phi \longrightarrow \{\theta_j\} \longrightarrow N_{\text{orient}}.$$

`--auto EPS` оставляет n фиксированным и подбирает N_ϕ, сетку θ и число точек Соболя. `--autofull EPS` дополнительно подбирает n. `--diffraction-autofull EPS` завершает поиск регулярной квадратурой Эйлера: N_β и N_γ уточняются по очереди, затем проверяются совместно. Цикл повторяется на увеличенном контрольном наборе, пока выбранные параметры не перестанут изменяться.

В совместном автоматическом режиме параметр `--phi-points N` фиксирует N_ϕ. Параметры `--scattering-grid` и `--theta-grid-file` фиксируют сетку θ; остальные параметры продолжают подбираться. Чтобы искать эту размерность, соответствующий явный флаг надо убрать. Отдельные `--adaptive-phi` и `--auto-phi` по-прежнему несовместимы с `--phi-points`, потому что их единственная задача — подобрать N_ϕ.

`--auto-theta-grid EPS` проверяет две внутренние точки каждого интервала по полной матрице Мюллера и по интегралу M_{11}. В алгоритме нет заранее заданных углов особых областей: особенности конкретной формы должны обнаруживаться по рассчитанной кривой. Приближённая стоимость равна

$$\text{стоимость} \approx N_{\text{ori}}\, N_\theta\, N_\phi\, N_b.$$

Поэтому автоматический режим может быть заметно дороже одной заранее заданной сетки, но он даёт измеряемый критерий выбора параметров.

Каждая попытка записывается в файл `<output>/<name>_convergence.tsv`: параметр, номер совместного прохода, кандидат, максимальная ошибка, худший theta-угол, номер элемента и статус.

12.10. Почему симметрия столбика не делит N_ϕ на шесть

В коде используется соглашение

$$\mathbf{R} = R_z(\alpha)R_y(\beta)R_z(\gamma).$$

Цикл по азимуту рассеяния за счет вращательной ковариантности реализует усреднение по лабораторному углу α, поэтому в этом контексте $N_\phi = N_\alpha$. У шестигранного столбика есть осевая симметрия $R_z(2\pi/6)$ в системе частицы. Она уже сокращает область γ до

$$0 \le \gamma < \frac{\pi}{3} = 60^\circ.$$

Но при $\beta \neq 0, \pi$ наклоненный столбик не инвариантен относительно поворота на 60° вокруг лабораторной оси падающего луча. Поэтому для α остается область $0 \le \alpha < 2\pi$, и автоматическое деление N_ϕ на шесть было бы физически неверным. `--alpha-points` является понятным алиасом `--phi-points`, а `--adaptive-alpha` — алиасом `--adaptive-phi`; они не включают дополнительное приближение.

12.11. Параллельная трассировка ориентаций из файла

`--parallel-trace` включает параллельную трассировку CPU PO для `--orientation-file`. Число потоков задаёт `--threads N`. Каждый поток получает собственную частицу, состояние трассировщика и буферы пучков. Без флага трассировка последовательная; дифракция сохраняет прежний параллельный код. При одном потоке используется последовательный путь. Каждая строка файла сохраняет равный вес. Амплитуды складываются внутри ориентации, затем усредняются матрицы Мюллера.

```
cat > orientations.txt <<'EOF'
# beta_deg gamma_deg
0 0
60 0
90 180
120 90
EOF
cpu/bin/mbs_po_mpi --method po --backend cpu \
  --particle 1 10 10 --refractive-index 1.31 0 --wavelength-um 0.532 \
  --orientation-file orientations.txt --parallel-trace --threads 4 \
  --scattering-grid 170 180 4 40 --max-reflections 6 \
  --cutoff-profile off --output results/file_average --close
```

Используйте один MPI-процесс. Сочетания с фиксированной ориентацией, GO и CUDA отклоняются. Отдельные буферы потоков увеличивают пиковую память. Порядок и веса в checkpoint сохраняются; продолжение возможно при смене последовательной трассировки на параллельную и обратно.

12.12. Повторное использование одинаковых ориентаций

Флаг `--deduplicate-orientations` объединяет только пары β, γ, точно равные после чтения файла. Близкие углы, симметрийно связанные узлы и разные γ на полюсе не объединяются. Если пара встречается c раз в N строках, её вес Мюллера равен c/N: исходная квадратура сохраняется. Оба флага независимы, выключены по умолчанию и требуют CPU PO из файла, одного MPI-процесса. Добавьте вторую строку `60 0` в предыдущий файл:

```
cpu/bin/mbs_po_mpi --method po --backend cpu \
  --particle 1 10 10 --refractive-index 1.31 0 --wavelength-um 0.532 \
  --orientation-file orientations.txt --parallel-trace \
  --deduplicate-orientations --threads 4 \
  --scattering-grid 170 180 4 40 --max-reflections 6 \
  --cutoff-profile off --output results/file_average --close
```

Пять строк потребуют четырёх трассировок с весами $1/5, 2/5, 1/5, 1/5$. Журнал показывает число строк и уникальных трассировок. Подпись checkpoint включает кратности и веса: продолжайте с тем же файлом и флагом объединения. Checkpoint с другим распределением весов не используется. На трёх частицах пыли совместные флаги дали около пятикратного ускорения той же квадратуры из 24 строк. Выигрыш и дополнительная память потоков зависят от задачи.

12.13. Аналитический физический компонент в точке назад

Флаг `--analytic-backscatter` вычисляет

$$\langle \widehat{M_{11}(180^\circ)} \rangle = \langle C \rangle + \sum_i w_i \, [M_{11}^{\mathrm{MBS}}(i) - C(i)].$$

C — профиль полос возврата: передача внутрь/наружу, два внутренних отражения в обоих порядках и когерентная сумма двух входов. Сохраняются угловой Френель, фазы полного внутреннего отражения и относительная оптическая фаза. Код находит прямые углы соседних граней и их параллельные противоположные грани. Прямоугольная и правильная гексагональная призмы дают по 12 полос. Для гексагона прямоугольная полоса приближает апертуру основания.

Среднее конечного профиля sinc^2 и фазовые моменты Лежандра/Bessel интегрируются аналитически. Гладкие физические коэффициенты готовятся численной одномерной квадратурой. Проверяются 96/160 гармоник, при необходимости 320; допустимое относительное изменение среднего — 10^{-6}. Никакая PDF не обучается вместо физического рассеяния. MBS сначала складывает все трассированные комплексные пучки, затем вычитает C из полной интенсивности. Длинные пути, изменения окон и межсемейная интерференция остаются в численном остатке.

```
cpu/bin/mbs_po_mpi --method po --backend cpu \
  --particle 1 101.02 101.02 --refractive-index 1.31 0 \
  --wavelength-um 0.532 --max-reflections 20 \
```

```
--sobol-seed 4096 42 --analytic-backscatter --threads 4 \
--scattering-grid 179 180 4 4 --cutoff-profile off \
--output results/analytic_backscatter --close
```

`--analytic-return-order K` добавляет собственные члены более длинных семейств с $p + q \leq K + 1$: p — проходы туда-обратно по нормали входа, $2q - 1$ — боковые отражения. Диапазон $K = 1 \dots 8$, по умолчанию $K = 1$; требуется `--max-reflections` не меньше $2K + 1$. Собственная фаза каждого члена сокращается; реберный интеграл аналитический, угловая амплитуда готовится одномерной квадратурой. Перекрёстные члены между семействами остаются в полном численном остатке. Например, добавьте `--analytic-return-order 3`. На проверенной призме 1:1 с $n = 1.31$ порядки 3–8 почти не изменили разброс: первый порядок остаётся default.

Используйте CPU PO, один MPI-процесс, вещественный $n > 1$ без поглощения, когерентную сумму, выпуклую частицу с подходящими рёбрами и точную строку 180°. Поддержаны Sobol, Sobol-seed, Hammersley, lattice и Euler quadrature. Симметрия самого C проверяется; при отказе задайте `--symmetry 1 1`. Multi-size, автоматические поиски сеток/глубины и mirror-gamma не поддержаны.

В обычном `.dat` исправляется только M_{11} строки 180°; остальные элементы и углы имеют обычную численную оценку. То же исправление применяется к no-shadow. Диагностические файлы:

```
<name>_analytic_backscatter.tsv
<name>_analytic_backscatter_samples.tsv
```

Они содержат обычное/гибридное средние, известное/выборочное средние C, остаток, сходимость и значения по ориентациям. При малой выборке оценка может быть отрицательной; обрезка внесла бы смещение.

Число трассировок при том же N сохраняется. Выигрыш возможен за счёт меньшего N при той же ошибке полного среднего; он не гарантирован. Проверяйте разброс по независимым Owen seeds. Скрипт `scripts/benchmark\_analytic\_backscatter.py` сравнивает обычное и гибридное средние на одной выборке.

12.14. Аналитические компоненты на всей сетке рассеяния

`--analytic-facet-average` исправляет только M_{11} на каждой строке θ. Усредняются собственные отражения от граней и теневые апертуры; все внутренние пути, затенение и перекрёстные члены остаются в полном когерентном численном остатке MBS. При том же N число трассировок сохраняется.

Косинус падения на грань $\mu = \vec{n} \cdot \vec{u}$ отличается от угла рассеяния $\theta = \arccos(-\vec{u} \cdot \vec{v})$. Внешняя нормаль $\vec{n}$ единичная; $\vec{u}$ направлен к источнику, $\vec{v}$ — к наблюдателю. При $\theta = 0$ наблюдение идёт вперёд. Френель зависит от каждого μ, включая наклонное падение. Общее вращение геометрии и обоих направлений сохраняет M_{11}. Векторный вызов `Control::Evaluate(u,v)` нормирует направления; `SourceFrame` и `Observer` строят произвольную ось падения и азимут на CPU/CUDA. Касание в пределах восьми eps FP64 имеет одинаковую обработку. В CLI все углы падения задаются поворотами частицы, дополнительный флаг не нужен. Полное среднее Haar независимо от лабораторной оси света; для ориентированного ансамбля это свойство неприменимо.

`--haar-alpha` добавляет третью Owen-Sobol координату для лабораторного α: первые β, γ точки прежнего Sobol сохраняются. Теперь допустимо $N_\phi = 1$; больше направлений дополнительно усредняют азимут. Альтернатива — `--so3-full-quaternion N`. Двухугловая выборка без α отклоняется: вне полюсов этого недостаточно для меры Haar.

```
cpu/bin/mbs_po_mpi --method po --backend cpu \
  --particle 1 30.3063 30.3063 --refractive-index 1.31 0 \
  --wavelength-um 0.532 --max-reflections 20 \
  --sobol-seed 4096 42 --haar-alpha --analytic-facet-average \
  --threads 4 --scattering-grid 0 180 1 36 \
  --cutoff-profile off --output results/analytic_all_angles --close
```

Пусть $\vec{u}$ направлен к источнику, $\vec{v}$ — к наблюдателю в системе частицы. Фазовые векторы различаются:

$$\vec{q}_r = k(\vec{u} + \vec{v}), \quad |\vec{q}_r| = 2k\sin(\theta/2), \qquad \vec{q}_s = k[\vec{v} - (\vec{u}\cdot\vec{v})\vec{u}], \quad |\vec{q}_s| = k\sin\theta.$$

Угловой Френель сохраняется. Теорема сложения Лежандра и разложение плоской волны интегрируют азимут и осциллирующую фазу; одномерные моменты $R(\mu)$ переиспользуются для всей сетки. Радиальная таблица Green и граничная квадратура остаются численными. Изменение отражённого компонента проверяется до 10^{-4}; малый теневой член нормируется суммарным компонентом. Используются до 320 гармоник и 768 узлов на ребро. Подготовка может быть заметной на больших частицах; её время записывается отдельно.

Одинаковые грани переиспользуют интегралы self-среднего; все нормали сохраняются в точечном компоненте и когерентной проецируемой площади. Две радиальные таблицы допускают по64 миллиона FP64-элементов (около1ГиБ вместе), в том числе для призмы высотой891.3мкм на сетке0–25°.

При 0° отражённый компонент нулевой. Среднее квадрата суммарной проецируемой площади вычисляется аналитически через все пары нормалей граней. Для выпуклой частицы это собственный вклад полного теневого пучка вперёд; невыпуклая проекционная модель исправляется полным остатком. При $n = 1 + 0i$ поправка выключена. Поглощение поддержано при $\mathrm{Re}\, n \geq 1$. Комплексный Френель компонента может слегка отличаться от существующей схемы MBS с реальным направлением преломленного луча; разница остаётся в остатке.

`--analytic-shadow-control facets|circular|off` выбирает тень. По умолчанию facets использует собственные проекции граней и когерентную площадь в 0°. Circular — круговая дифракционная апертура с радиусом из средней проекционной площади и площадью исходной ориентации; это аналитический приближённый компонент с исправлением в остатке. Off оставляет только отражение. При no-shadow теневая поправка выключена; no-shadow-output получает только отражение.

Диагностические файлы:

```
<name>_analytic_facets.tsv
<name>_analytic_facet_samples.tsv
<name>_analytic_combined.tsv
```

Первый содержит raw/hybrid, известные/выборочные средние, остаток и сходимость. Второй включается `--analytic-facet-samples`; по умолчанию объёмный поориентационный вывод выключен. Третий показывает совместный итог при `--analytic-backscatter`: возвратная поправка добавляется только в 180°. Отдельные summary каждого компонента сохраняют собственную оценку от исходного MBS.

Условия: CPU/CUDA PO, один MPI rank, coherent, один размер, фиксированные сетки и глубина, полная мера Haar. Форма может быть произвольной гранёной; при неподтверждённой симметрии задайте `--symmetry 1 1`. Отношение дисперсий в проверке трёх призм на 8×512 ориентациях: примерно 0.77–1.52. Некоторым углам поправка не помогла. Это не измеренное ускорение до допуска и не аналитическое закрытие всех внутренних путей.

`--analytic-mean-cache FILE` переиспользует проверенные средние для одинаковых геометрии, материала, длины волны и сетки, с блокировкой процессов и атомарной записью. Поориентационные апертуры вычисляются в FP64 на CUDA; полное когерентное поле сохраняет прежний оптимизированный GPU-путь.

Переносимый `scripts/run\_gpu\_multilevel.py` использует

$$E[M_f] = E[M_c] + E[M_f - M_c].$$

Большая выборка coarse сочетается с меньшей парной выборкой поправки. Матрицы Мюллера строятся после полного когерентного сложения пучков каждой ориентации. Восемь итоговых оценок по seeds сохраняют корреляцию компонент. По умолчанию coarse — depth8/cutoff0.001, fine — depth12/cutoffoff; два последовательных изменения и интервалы ошибки должны быть меньше3% во всех строках. Все элементы нормируются на M_{11}; исходные GPU-данные сохраняются.

```
python3 scripts/run_gpu_multilevel.py \
  --binary gpu/bin/mbs_po_gpu_double --case test2 \
  --output results/test2 --gpus 0 1 2 3 --threads 16 --phi-points 4
```

Пресеты воспроизводят призмы446.7/138.9мкм и891.3/207.8мкм, показатель1.3116, длину волны0.532мкм и267/531строку на0–25°. Физические входы и theta-файл переопределяются; `--dry-run` показывает параметры. `status.json` содержит прогресс, `mueller\_multilevel.dat` — итог. При повторе проверяются бинарник и физические параметры; после аварии контроллера проверяйте PID заданий перед повторным запуском.

Экспериментальный `--analytic-azimuth-gaussian` добавляет к `--analytic-facet-average` компоненты внутренних пучков. Проецируемая площадь A и фактическая норма Jones задают $P = \|J\|^2 A^2/(2\lambda^2)$, $\kappa = 2\pi A/\lambda^2$. Для полярного угла пучка θ_b:

$$C = Pe^{-\kappa|\vec{d}-\vec{v}|^2/2}, \qquad E_\alpha C = Pe^{-2\kappa \sin^2[(\theta-\theta_b)/2]} I_{0e}(\kappa \sin\theta \sin\theta_b),$$

где $I_{0e}(x) = e^{-x} I_0(x)$, см. NIST DLMF 10.32.1. Это точное условное среднее гауссова компонента; поправка $E_\alpha C - C$ сохраняет всю полигональную дифракцию и интерференцию в остатке MBS. Полярное окно с экспонентой64 не зависит от α и применяется к обоим членам. При $n = 1$ и в обеих полюсных точках поправка нулевая. Файл `\_analytic\_azimuth.tsv` содержит значения и время; подробные выборки включаются `--analytic-facet-samples`. На некоторых

углах дисперсия может расти; проверяйте независимые seeds перед рабочим расчётом. Системы координат готовятся по пакетам: экономится $72N$ байт памяти, а генерация Haar не хранит ненужные двумерные точки, экономя ещё $16N$ байт.

13. Угловая сетка рассеяния

13.1. Строки по θ и телесный угол

Равномерная сетка `--scattering-grid T0 T1 NPHI NTH` содержит $N_\theta + 1$ строк. Вес строки равен

$$\Delta\Omega_j = 2\pi \left[\cos\theta_{j,\text{left}} - \cos\theta_{j,\text{right}}\right].$$

Для полного диапазона $0^\circ \ldots 180^\circ$:

$$\sum_j \Delta\Omega_j = 4\pi.$$

13.2. Азимутальная сетка

В полном расчёте используется N_ϕ азимутальных точек. В итоговом файле для случайных ориентаций обычно записан результат, уже усреднённый по азимуту, но внутри алгоритма ϕ остаётся отдельным измерением работы:

$$N_{\text{work}} \sim N_b\, N_\theta\, N_\phi\, N_{\text{ori}}.$$

13.3. Дифракционная оценка и широтная сетка по ϕ

В PO-режиме `--diffraction-sampling Q` использует для сетки рассеяния тот же масштаб $\xi_0 = 0.69\lambda/l_{\max}$, что и для ориентаций. Целевой шаг равен

$$\Delta_{\text{scat}} = \frac{\xi_0}{Q_{\text{scat}}}.$$

Числа интервалов вычисляются как

$$N_\theta = \left\lceil \frac{\pi}{\Delta_{\text{scat}}} \right\rceil,$$

$$N_{\phi,\text{eq}} = \text{round_up}_6\left[\max\left(12, \left\lceil \frac{2\pi}{\Delta_{\text{scat}}} \right\rceil\right)\right].$$

Результат содержит $N_\theta + 1$ строк от 0 до π. Значение $Q = 1$ использует дифракционный масштаб непосредственно, а $Q = 2$ уменьшает оба целевых угловых шага вдвое. Это априорная оценка разрешения, а не гарантия точности. Для итогового расчёта следует сравнить выбранное Q с удвоенным значением по всем требуемым элементам матрицы Мюллера.

Параметр `--scattering-diffraction-sampling Q` переопределяет только Q_{scat}. Например,

```
--diffraction-sampling 2 \
--scattering-diffraction-sampling 1
```

оставляет шаг ориентаций $\xi_0/2$, но задаёт шаг рассеяния ξ_0.

Флаг `--latitude-phi-grid` сохраняет экваториальное разрешение, но задает отдельное число азимутальных точек для каждой строки:

$$N_\phi(\theta) = \min\left\{ N_{\phi,\mathrm{eq}}, \mathrm{round_up}_6\left[\max\left(12, \left\lceil \frac{2\pi \sin\theta}{\Delta_{\mathrm{scat}}} \right\rceil \right) \right] \right\}.$$

Так удаляются почти совпадающие азимутальные направления около полюсов без огрубления экватора. Число узлов округляется до кратного шести, а полюсная строка связывается с соседней рабочей группой для устойчивой обработки базиса. Накопление выполняется на полной прямоугольной выходной сетке $N_{\phi,\mathrm{eq}}$, поэтому формат результата не меняется.

Переменная широтная сетка поддерживается как для дифракционной сетки ориентаций, так и для сокращённого режима `--so3-quaternion N`, по одному размеру частицы на процесс. Для SO(3) предпочтителен параметр `--scattering-diffraction-sampling Q`: общий `--diffraction-sampling Q` сам задаёт основной регулярный ориентационный режим. Автоматическая сетка конфликтует с явными `--scattering-grid`, `--theta-grid-file`, `--auto-theta-grid` и `--phi-points`. Широтная сетка пока не поддерживается общим последовательным расчётом нескольких размеров. Каждый размер следует запускать отдельным процессом, чтобы $l_{\max}$ и сетка пересчитывались заново.

```
CUDA_VISIBLE_DEVICES=0,1,2,3 \
MBS_GPU_GROUPS=1 MBS_GPU_MULTI_MAX=4 \
gpu/bin/mbs_po_gpu_double --method po \
  --particle 1 100 70 --refractive-index 1.3116 0 \
  --wavelength-um 0.532 --max-reflections 8 \
  --diffraction-sampling 2 \
  --latitude-phi-grid --backend cuda --threads 32 \
  --output results/column --close
```

13.4. Адаптивная сетка по θ

Передний дифракционный пик обычно требует более мелкого шага, чем боковые и задние углы. Режим `--auto-theta-grid EPS` не использует заранее назначенные зоны. Он сравнивает рассчитанные значения с интерполяцией в точках 1/3 и 2/3 каждого интервала, уточняет только не прошедшие интервалы и отдельно контролирует изменение интеграла M_{11}. Верхняя

граница числа узлов задаётся параметром `--max-theta-points`.

Часть IV

Реализация и производительность

14. Реализация на CPU и AVX

14.1. Что параллелится на CPU

В CPU-версии OpenMP и MPI распределяют ориентации, строки сетки θ, пучки и группы размеров. К наиболее затратным операциям относятся вычисление фаз

$$\Phi = k(A_q x + B_q y), \qquad \exp(\mathrm{i}\Phi) = \cos\Phi + \mathrm{i}\sin\Phi,$$

суммирование вкладов рёбер и накопление матриц Джонса и Мюллера.

Для регулярной фазы многоугольника CPU-цикл PO обрабатывает четыре соседних направления θ одной AVX2-операцией над интегралом по рёбрам. В каждой векторной полосе сохраняется исходный порядок суммирования рёбер. При малой фазе или активном почти сингулярном знаменателе вся группа автоматически переходит на исходную устойчивую скалярную формулу. Режим включён в обычную CPU-сборку, не требует флага и не вводит новой пользовательской точности. Временные массивы фазы и синуса/косинуса хранятся сначала по вершине, затем по соседним направлениям θ. Поэтому AVX2-цикл загружает четыре соседних значения напрямую вместо сбора из четырёх строк. На представительном тесте поглощающего вогнутого столбика медианное время уменьшилось с 12,76 до 11,37 с (в 1,12 раза) относительно предыдущей векторизованной реализации, а таблица Мюллера совпала побитно.

14.2. Упаковка данных в AVX-512

Для AVX-512 в двойной точности:

$$\mathbf{\Phi} = (\Phi_0, \Phi_1, \ldots, \Phi_7),$$

$$\mathbf{C} = \cos\mathbf{\Phi}, \qquad \mathbf{S} = \sin\mathbf{\Phi}.$$

Одна векторная команда или векторизованный полиномиальный алгоритм обрабатывает сразу восемь направлений наблюдения либо восемь размеров. Для AVX2:

$$\mathbf{\Phi}_{\mathrm{AVX2}} = (\Phi_0, \Phi_1, \Phi_2, \Phi_3).$$

14.3. Почему важны флаги компиляции

Если компилятор не имеет права генерировать AVX-512, быстрый путь не включится. Для EPYC:

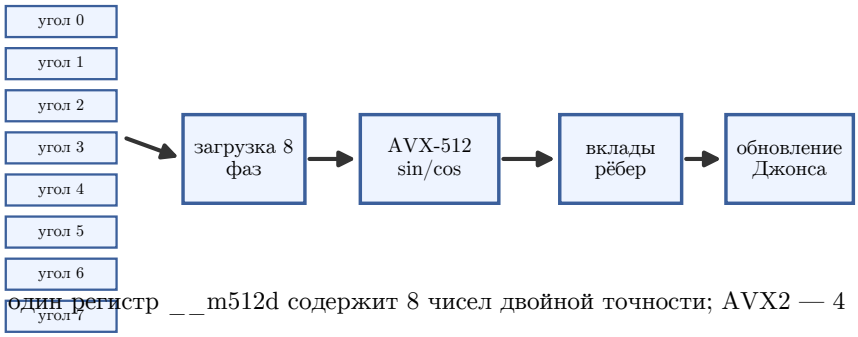


Рис. 7: AVX-512 обрабатывает восемь значений двойной точности в одном векторном регистре; AVX2 обрабатывает четыре.

Таблица 15: Рекомендуемые CPU-флаги.

Платформа	`ARCH_FLAGS`	Комментарий
EPYC Zen2	`-march=znver2 -mtune=znver2`	AVX2/FMA.
EPYC Zen3	`-march=znver3 -mtune=znver3`	AVX2/FMA.
EPYC Zen4	`-march=znver4 -mtune=znver4`	AVX-512 при поддержке GCC/Clang.
EPYC Zen5	`-march=znver5 -mtune=znver5`	Лучший вариант для новых компиляторов.
Переносимый AVX2	`-O3 -mavx2 -mfma`	Для переносимого бинарного файла под AVX2.
Явный AVX-512	`-O3 -mavx512f -mavx512dq -mavx512cd -mavx512bw -mavx512vl`	Только если `lscpu` показывает эти возможности.

Флага `AVX12` у GCC/Clang нет. Если имеется в виду EPYC Zen5 с AVX-512, используйте `-march=znver5`; если компилятор старый, используйте явные `-mavx512*` после проверки `lscpu`.

15. Реализация на GPU

15.1. Что ускоряет GPU

GPU ускоряет дифракционную часть, потому что она раскладывается на почти независимые элементы работы:

$$(b, \theta, \phi, o) \mapsto \mathbf{J}_{b,o}(\theta, \phi),$$

где b — номер пучка, а o — номер ориентации. До когерентного суммирования каждый вклад вычисляется независимо.

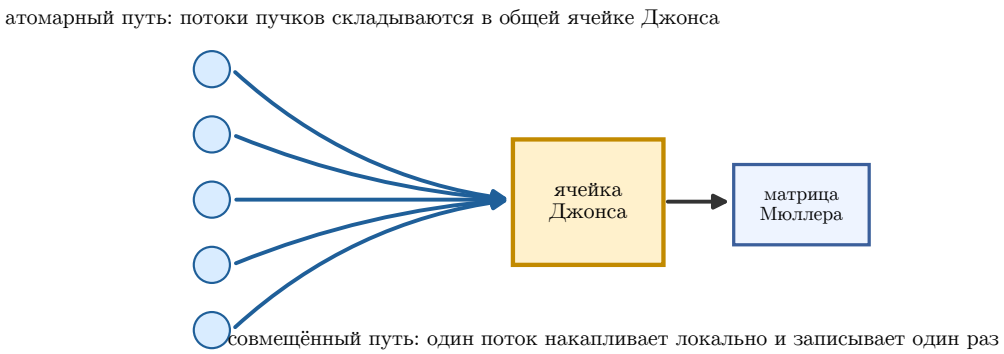


Рис. 8: Варианты CUDA-вычислений: атомарное накопление матриц Джонса, владение ячейкой без атомарных операций и совмещённое вычисление дифракции с преобразованием в матрицу Мюллера.

Таблица 16: Основные варианты ядер GPU.

Вариант	Работа одного потока	Область применения
Атомарное накопление	Вычисляет вклад пучка и добавляет его командой `atomicAdd` в матрицу Джонса.	Универсальный резервный и диагностический путь.
Владение ячейкой	Один поток владеет выходной ячейкой и последовательно проходит пучки.	Когда размещение данных позволяет избежать конкурирующих записей.

Вариант	Работа одного потока	Область применения
Совмещённое ядро	Локально суммирует матрицу Джонса и сразу преобразует её в матрицу Мюллера.	Основной путь при записи только полного результата.
Поэтапное накопление	Сначала строит матрицы отдельных ориентаций, затем редуцирует их.	Большие блоки ориентаций и высокая конкуренция атомарных операций.
Несколько k_{eq}	Обрабатывает близкие размеры из общего набора пучков.	Серии размеров с совместимыми геометрией и сеткой.

15.2. Атомарное накопление

Матрица Джонса имеет четыре комплексных элемента:

$$\mathbf{J} = \begin{pmatrix} J_{00} & J_{01} \\ J_{10} & J_{11} \end{pmatrix}, \qquad J_{pq} = x_{pq} + \mathrm{i}y_{pq}.$$

Атомарный вариант выполняет восемь сложений:

$$\mathrm{atomicAdd}(x_{pq}), \qquad \mathrm{atomicAdd}(y_{pq}), \qquad p, q \in \{0, 1\}.$$

Это корректно для когерентной суммы, но порядок сложения не детерминирован. Поэтому младшие разряды могут отличаться от CPU, особенно в одинарной точности и около вырожденных направлений.

15.3. Почему совмещённое ядро быстрее

Совмещённое ядро не записывает полный промежуточный массив матриц Джонса в глобальную память:

$$\sum_b \mathbf{J}_b \;\longrightarrow\; \mathcal{M}\left(\sum_b \mathbf{J}_b\right) \;\longrightarrow\; \text{глобальный массив Мюллера.}$$

Это уменьшает обмен с глобальной памятью и число атомарных операций, повышая вычислительную интенсивность. Эффект особенно заметен при больших N_b и N_ϕ.

Отдельное ускорение по азимуту даёт интерполяция cuFFT. Для автоматического выбора используется `--fft`, для явного и воспроизводимого сжатия `--fft-factor N`, а для измеряемого контроля на вложенной сетке `--fft-tolerance EPS`. Алгоритм, цена дополнительной проверки и смысл оценки ошибки разобраны в разделе 22.3.

15.4. Автоматический профиль трассировки на GPU

В CUDA-сборке метода физической оптики пакетное упорядочивание граней и консервативный отбор кандидатов включаются автоматически. Явный параметр `--gpu-trace-prefilter`

включает тот же режим, а `--no-gpu-trace-prefilter` отключает его для контрольного сравнения. Это не приближённая трассировка: точные многоугольники пересечения, топологические решения, коэффициенты Френеля и построение дочерних пучков остаются на CPU.

Проверенный автоматический профиль включает точную топологическую сортировку на CUDA, кэш одинаковых сортировок, консервативные флаги пропуска для списков из 25–256 граней, кэш геометрии граней и блок из 64 потоков для больших списков. Каждый рабочий поток OpenMP использует отдельный неблокирующий поток CUDA. Размер пакета одного рабочего потока выбирается как

$$B = \max\left(64, \left\lceil \frac{512}{N_{\text{workers}}} \right\rceil\right).$$

Такая формула предотвращает исчерпание стека CUDA, наблюдавшееся при завышенном фиксированном пакете. Если и начальный пакет исчерпывает стек CUDA, он рекурсивно делится, а уменьшенный предел запоминается для всех следующих пакетов и ориентаций процесса. При включённых повторах рабочие копии также совместно запоминают наибольший успешно проверенный предел числа пучков. Следующие ориентации пропускают заведомо недостаточный первый проход, но число уровней по-прежнему отсчитывается от исходного предела, поэтому верхняя граница не расширяется. На тесте из 32 ориентаций время изменилось с 11,12 до 8,00 с, а число полных повторов с 30 до 7. Неблокирующие потоки изменили время 256 ориентаций с 61,39 до 42,26 с. Выходные таблицы и интегральные характеристики в обоих A/B-тестах совпали побитово. Независимые процессы на одной физической видеокарте последовательно выполняют только тяжёлый пакет трассировки через блокировку по PCI-адресу. Рабочие потоки OpenMP внутри одного процесса сохраняют независимые потоки CUDA и перекрываются. Это предотвращает исчерпание памяти суммарным резервом стека CUDA-ядер и не блокирует ядра дифракции. На сложной ориентации глубоко вогнутой частицы время уменьшилось с 7,93 до 0,98 с, то есть в 8,09 раза; на более лёгком контрольном случае выигрыш составил около 1,08 раза. Эффект зависит от сложности дерева трассировки.

Таблица 17: Параметры автоматической CUDA-трассировки.

Переменная	По умолчанию	Смысл
`MBS_GPU_TRACE_OPENMP`	1	Вызывать CUDA-трассировку из рабочих потоков OpenMP.
`MBS_GPU_TRACE_BATCH_BEAMS`	авто	Переопределить начальный пакет; после OOM предел уменьшается и запоминается.

Переменная	По умолчанию	Смысл
MBS_GPU_TRACE_INFLIGHT_BEAMS	512	Ограничить суммарное число пучков CUDA-трассировки всех OpenMP-потоков; после OOM предел уменьшается.
MBS_GPU_TRACE_FULL_SORT	1	Точная топологическая сортировка на CUDA.
MBS_GPU_TRACE_SORT_CACHE	1	Повторно использовать сортировку одинаковых ключей.
MBS_GPU_TRACE_LARGE_SKIP_FLAGS	1	Консервативные флаги пропуска для больших списков.
MBS_GPU_TRACE_LARGE_THREADS	64	Размер блока: 64, 128 или 256.
MBS_GPU_TRACE_CACHE_FACETS	1	Хранить упакованную геометрию граней в рабочей области.
MBS_GPU_TRACE_MIN_CANDIDATES	1024	Минимум кандидатов для отправки пакета на GPU.
MBS_GPU_TRACE_NONBLOCKING_STREAM	1	Неблокирующий поток CUDA для каждого worker; 0 включает последовательную диагностику.
MBS_GPU_TRACE_PROCESS_LOCK	1	Последовательно выполнять тяжёлые пакеты трассировки независимых процессов на одной физической GPU.
MBS_GPU_TRACE_PREFILTER_FIRST	0	Диагностический грубый отбор перед сортировкой.
MBS_GPU_TRACE_MARGIN	авто	Переопределить консервативный запас проекционных границ.
MBS_GPU_TRACE_TIMING	0	Печатать времена передачи и ядер.

Переменная	По умолчанию	Смысл
MBS_GPU_TRACE_VERIFY_SORT	0	Сверять сортировку и флаги пропуска с CPU.

В проверке производственной частицы расхождений сортировки и ошибочных пропусков не обнаружено. Однако эквивалентный порядок обхода граней может немного изменить порядок сложения чисел с плавающей точкой. Поэтому для чувствительных расчётов выполняют выборочный контроль с --no-gpu-trace-prefilter; такой контроль входит в проверку релиза.

15.5. Работа на нескольких GPU

При работе на нескольких GPU независимыми единицами распределения служат ориентации, группы строк θ или серии размеров. Идеальная оценка времени на G устройствах имеет вид

$$T_G \approx \frac{T_1}{G} + T_{\text{copy}} + T_{\text{reduce}}.$$

Линейному ускорению мешают обмены PCIe/NVLink, неодинаковая скорость устройств, размер блоков и финальное сложение матриц Мюллера.

Для переменной широтной сетки по ϕ MBS_GPU_GROUPS=1 включает отдельный точный планировщик. Независимые группы строк θ назначаются видимым GPU динамически. Трассировка ориентаций и подготовленные пучки остаются общими в оперативной памяти; на каждой карте создаётся одна рабочая область CUDA. В пределах блока ориентаций упакованные пучки, веса и смещения загружаются один раз и повторно используются следующими группами строк.

Набор карт выбирается через CUDA_VISIBLE_DEVICES. Число исполнителей задаёт MBS_GPU_MULTI=N; значение 0 оставляет одну карту. Дополнительную верхнюю границу задаёт MBS_GPU_MULTI_MAX=N. Планировщик применяется только к прямой CUDA-дифракции без FFT-интерполяции и фильтрации пучков по зонам θ. Ускорение не обязано быть линейным: трассировка на CPU, упаковка, копирование по PCIe, редукция и дисбаланс групп θ остаются в полном времени.

Все активные переменные MBS_* записываются в итоговый журнал. Переменные, меняющие выборку, геометрию, отсечения или численный результат, запрещены без явного флага --allow-experimental-environment. Для блоков ориентаций программа начинает с 16 пробных ориентаций, измеряет фактическую память вложенных пучков и траекторий и затем подбирает размер блока по текущей доступной ОЗУ.

Таблица 18: Переменные окружения для GPU.

Переменная	Смысл
CUDA_VISIBLE_DEVICES=0,1	Выбрать доступные GPU.
MBS_GPU_FUSED_MUELLER=1	Предпочитать совмещённое ядро дифракции и преобразования Мюллера.
MBS_GPU_STAGE_MUELLER=1	Включить поэтапное накопление по ориентациям.

Переменная	Смысл
MBS_GPU_COMPACT_BEAMS=0	Отключить компактную упаковку пучков с не более чем 8 вершинами.
MBS_GPU_COMPACT_BEAM4_SPLIT=0	Отключить отдельный четырёхвершинный формат треугольников и четырёхугольников.
MBS_GPU_WARP_BEAMS=0/1	Переопределить автоматический выбор: один CUDA-поток на выходную точку (0) или один warp (1); для некогерентного режима автоматика учитывает размер сетки и число пучков.
MBS_GPU_WARP_GRID_3D=0	Отключить специализацию трёхмерной сетки при активной warp-редукции.
MBS_GPU_THREAD_GRID_3D=0/1	Переопределить плиточную трёхмерную сетку потокового пути; автоматика включает её для блока 64, если краевые плитки добавляют не более 15% работы.
MBS_GPU_TRACE_OPENMP=0/1	Включить или отключить автоматическую CUDA-трассировку из потоков OpenMP.
MBS_GPU_TRACE_BATCH_BEAMS=N	Переопределить автоматический размер пакета одного потока.
MBS_GPU_TRACE_INFLIGHT_BEAMS=N	Ограничить суммарное число одновременно трассируемых пучков процесса; по умолчанию 512, минимум 32.
MBS_GPU_TRACE_FULL_SORT=0/1	Точная топологическая сортировка CUDA; по умолчанию 1.
MBS_GPU_TRACE_SORT_CACHE=0/1	Кэш точной сортировки; по умолчанию 1.
MBS_GPU_TRACE_LARGE_SKIP_FLAGS=0/1	Консервативные флаги пропуска больших списков; по умолчанию 1.
MBS_GPU_TRACE_LARGE_THREADS=N	Размер блока большого ядра; по умолчанию 64.
MBS_GPU_TRACE_CACHE_FACETS=0/1	Кэшировать упакованную геометрию граней; по умолчанию 1.
MBS_GPU_TRACE_MIN_CANDIDATES=N	Минимум кандидатов в отправляемом пакете; по умолчанию 1024.
MBS_GPU_TRACE_NONBLOCKING_STREAM=0/1	Неблокирующие потоки CUDA; по умолчанию 1; 0 включает последовательную диагностику.
MBS_GPU_TRACE_PROCESS_LOCK=0/1	Межпроцессная блокировка тяжёлых пакетов трассировки на физической GPU; по умолчанию 1.
MBS_GPU_TRACE_TIMING=1	Печатать времена CUDA-трассировки.
MBS_GPU_TRACE_VERIFY_SORT=1	Сверять CUDA-сортировку с CPU.
MBS_GPU_MULTI=0	Отключить автоматическое распределение блоков ориентаций.
MBS_GPU_MULTI_MAX=N	Ограничить число исполнителей GPU.

Переменная	Смысл
`MBS_GPU_GROUPS=1`	Распределить группы θ широтной сетки по видимым GPU.
`MBS_GPU_MULTI_K_FULL=0/1`	Совмещённый расчёт размеров: автоматически для поддержанных FP64-сканов, явно для float.
`MBS_ORIENTATION_TIMING=1`	Печатать суммарное процессорное время поворота, трассировки и подготовки пучков.
`MBS_FFT_PHI_FACTOR=N`	Старый способ задания коэффициента FFT, только если не задан `--fft-factor`; CLI имеет приоритет.
`MBS_SHARED_ORIENT_CHUNK=N`	Переопределить размер блока ориентаций.
`MBS_BEAM_LOG=PATH`	Записать первый обработанный набор PO-пучков; по умолчанию такой журнал не создаётся.

В когерентном режиме CUDA использует сообщаемое устройством отношение производительности FP32 к FP64. При отношении 16 и выше выбирается один поток на выходную точку, при меньшем отношении сохраняется warp-редукция. В некогерентном режиме один warp на выходную точку выбирается при наличии не менее 32 пучков на ориентацию и не более 200,000 сочетаний «ориентация–точка сетки»; в остальных случаях используется один поток на точку. Общая скалярная фаза исключается точно по тождеству $\mathrm{Mueller}(cJ) = |c|^2\,\mathrm{Mueller}(J)$, а в выбранной точности GPU вычисляется только локальная фаза апертуры. Когерентное суммирование матриц Джонса сохраняет точное вычисление фазы. При блоке 64 потоковый путь также использует плитку $16\,\theta \times 4\,\phi$, если неполные краевые плитки добавляют не более 15% работы. Это убирает два целочисленных деления на выходной поток; в проверке полное время изменилось на 0–11% в сторону ускорения. Переменная `MBS_GPU_TIMING=1` выводит `reduction=thread|warp` и `layout=3d|flat`.

В компактном потоковом пути автоматика отдельно упаковывает многоугольники с числом вершин не более четырёх и многоугольники с пятью–восемью вершинами. Более крупные многоугольники остаются в общем формате на 32 вершины; одно совмещённое ядро обрабатывает все три формата. Если у всех компактных пучков не менее трёх вершин, циклы треугольников и четырёхугольников имеют известную при компиляции длину. Один устойчивый проход по корзинам заменяет повторное сканирование списка для каждого числа вершин. Дополнительный флаг не нужен: это автоматический путь GPU по умолчанию.

На проверке вогнутой поглощающей частицы на RTX 3080 Ti разделение уменьшило суммарное время FP32-ядер CUDA примерно на 4%, а время всей GPU-дифракции на 3%. Для агрегата с выходными многоугольниками из 3–10 вершин время FP32-ядра уменьшилось на 13%, всей GPU-дифракции на 14%; для FP64 выигрыш составил 4% и 6% соответственно. Полный выигрыш меньше, когда преобладает неизменённая CPU-трассировка. Выпускной тест сверяет полностью компактный и смешанный пути с `MBS_GPU_COMPACT_BEAM4_SPLIT=0` для

обеих точностей.

Часть V

Сборка, запуск и данные

16. Сборка и исполняемые файлы

Для CPU требуются GNU Make, компилятор GCC 9 или Clang 14 и новее с OpenMP, а также согласованная пара MPI-компилятора и запускающего процесса. Для GPU дополнительно нужны драйвер NVIDIA, набор CUDA с `nvcc` и библиотека cuFFT. После сборки следует сохранить вывод `--version`: в нём указаны версия исходного кода и включённые возможности CPU/CUDA.

16.1. CPU: MPI и OpenMP

```
make -C cpu -j
make test_release
cpu/bin/mbs_po_mpi --version
```

По умолчанию CPU-версия оптимизируется под текущий процессор. Для переносимого бинарного файла архитектуру задают явно:

```
make -C cpu clean
make -C cpu ARCH_FLAGS='-march=x86-64-v3 -mtune=generic' -j
```

16.2. GPU: точность и быстрые математические функции

```
make gpu_fp64 -j
make gpu_fp32 -j
make gpu_fp32_consumer -j
make gpu_fp32_fast -j
make gpu_fp64_fast -j
make gpu_double_fast_mpi -j
```

Таблица 19: Основные варианты сборки GPU.

Цель	Исполняемый файл	Назначение
`gpu_fp64`	`gpu/bin/mbs_po_gpu_double`	Контрольный и основной FP64 без `--use_fast_math`.
`gpu_fp32`	`gpu/bin/mbs_po_gpu_float`	FP32 без `--use_fast_math`; требует проверки ошибки.
`gpu_fp32_consumer`	`gpu/bin/mbs_po_gpu_float_consumer`	Быстрый профиль потребительской GPU с FP32-тригонометрией фазы.

Цель	Исполняемый файл	Назначение
`gpu_fp32_fast`	`gpu/bin/mbs_po_gpu_float_fast`	Экспериментальный профиль; применять после проверки ошибки и времени.
`gpu_fp64_fast`	`gpu/bin/mbs_po_gpu_double_fast`	FP64 с CUDA `--use_fast_math`; требует проверки ошибки.
`gpu_double_fast_mpi`	`gpu/bin/mbs_po_gpu_mpi_double_fast`	FP64 с MPI и CUDA для распределённого запуска.

Низкоуровневая переменная сборки имеет вид `GPU_PHASE_TRIG=precise|consumer`. Значение `consumer` допустимо только с FP32-хранилищем, использует отдельный каталог объектных файлов и не включает `--use_fast_math`. Обычная команда — `make -C gpu fp32_consumer -j`. После сборки на целевой GPU выполните `make test_cuda_consumer`.

FP32 применяется к хранению геометрии дифракционных пучков, сумм Джонса и результата Мюллера. Трассировка видимости и топологии, оптические пути, поглощение, чувствительные к сокращению моменты многоугольников и общая фаза центра пучка остаются FP64. Точный FP32-бинарный файл строит и вычисляет фазы вершин в FP64. Потребительский профиль строит зависящую от координат часть в FP32, добавляет общую FP64-фазу в итоговый аргумент и применяет `sincosf`. Команда `--version` отдельно выводит точность построения фазы и её тригонометрии, профиль функций и архитектуру GPU.

После каждой пересборки перед запуском очереди проверьте профиль:

```
gpu/bin/mbs_po_gpu_double --version  # fp64-diffraction-storage ... precise-math
gpu/bin/mbs_po_gpu_float --version   # fp32-diffraction-storage ... precise-math
gpu/bin/mbs_po_gpu_float_consumer --version  # phase-build-mixed phase-trig-fp32
```

Управляющая C++-часть CUDA-сборки требует ровно один профиль FP32/FP64. Бинарный файл со строкой `unknown-precision` является устаревшим или собранным в обход поддерживаемых профилей и не должен использоваться для расчётов.

На RTX 3080 Ti для характерного поглощающего вогнутого теста ($L = 20$ мкм, 192 ориентации SO(3), $N_\phi = 288$, $N_\theta = 144$) получены следующие медианные времена:

Таблица 20: Измеренное сравнение профилей точности.

Профиль	Время	Ускорение	Ошибка Мюллера относительно FP64
Точный FP64	8,56 с	1,00	контрольный результат
Точный смешанный FP32	5,76 с	1,49	общая $L_2 = 5{,}68 \cdot 10^{-8}$
Потребительский FP32	3,25 с	2,63	общая $L_2 = 2{,}82 \cdot 10^{-7}$

После калибровки для потребительской GPU рекомендуется профиль `gpu_fp32_consumer`. Он в 1,77 раза быстрее точного смешанного FP32. В проверках выпуклой и вогнутой геометрии, поглощения, размеров $L = 5 \ldots 500$ мкм и 8–20 отражений максимальная L2-ошибка M11

в боковой и задней областях составила $8{,}4 \cdot 10^{-5}$; для одного слабого внедиагонального элемента получено $5{,}3 \cdot 10^{-3}$. Общий fast math был примерно на 9% медленнее потребительского профиля. Контрольные точки следует выборочно пересчитывать в FP64.

Смешанное построение фазы дополнительно проверено на RTX 3080 Ti для обычного и полого столбиков, агрегата и пулевой розетки вплоть до $L = 1000$ мкм. Относительно прежнего построения всей фазы в FP64 при том же `sincosf` время ядра дифракции уменьшилось на 4,6–8,3%, всего этапа GPU на 4,4–7,9%. Максимальное глобальное изменение матрицы Мюллера составило $4{,}3 \cdot 10^{-8}$, а максимальное относительное изменение точно при 180° — $3{,}7 \cdot 10^{-5}$. Полностью FP32-построение было быстрее, но отклонено: для пулевой розетки его ошибка при 180° достигла 0,68%.

Слово `fast` относится к параметру компилятора CUDA, а не к разрядности. По умолчанию `make -C gpu` собирает контрольный FP64-бинарный файл без быстрых математических функций; эквивалентная явная команда:

```
make -C gpu fp64 -j
```

Новый класс частиц сначала проверяют в CPU FP64 и GPU FP64 без быстрых математических функций. Варианты `double_fast` и `float_fast` допускаются после измерения ошибки на характерных размерах, ориентациях и направлениях 0° и 180°.

17. Запуск и первичная диагностика

17.1. Рабочий запуск на GPU

```
CUDA_VISIBLE_DEVICES=0 \
gpu/bin/mbs_po_gpu_float \
  --method po --particle 1 125.9 78.09 \
  --refractive-index 1.3116 0 --wavelength-um 0.532 \
  --max-reflections 12 --orientation-diffraction-sampling 2 \
  --scattering-grid 0 180 600 180 \
  --backend cuda --threads 16 \
  --output results/hex_gpu_double --close
```

17.2. Контроль CPU в том же GPU-бинарном файле

```
gpu/bin/mbs_po_gpu_double \
  --method po --particle 1 125.9 78.09 \
  --refractive-index 1.3116 0 --wavelength-um 0.532 \
  --max-reflections 12 --orientation-diffraction-sampling 2 \
  --scattering-grid 0 180 600 180 \
  --backend cpu --threads 64 \
  --output results/hex_cpu_same_binary --close
```

Такое сравнение исключает различие интерфейса и настроек двух отдельных сборок. Для окончательного контроля его дополняют запуском `cpu/bin/mbs_po_mpi`.

17.3. Проверка направлений 0° и 180°

```
gpu/bin/mbs_po_gpu_double \
  --method po --particle 1 125.9 78.09 \
  --refractive-index 1.3116 0 --wavelength-um 0.532 \
  --max-reflections 1 --orientation-diffraction-sampling 2 --pole \
  --scattering-grid 0 180 600 1 \
  --backend cuda --threads 16 \
  --output results/check_0_180 --close
```

18. Параметры командной строки

В таблицах ниже приведены канонические имена из `--help`. Сокращения `--po`, `-p`, `--pf`, `--ri`, `-w`, `-n`, `--oldauto`, `--diffraction-grid`, `--diffraction-limit-grid`, `--ring-points`, `--grid`, `--fixed`, `--random` и `-o` продолжают приниматься для старых сценариев, но в новых командах их использовать не следует. Полный перечень диагностических и экспериментальных параметров выводит `--help-debug`.

18.1. Метод, частица и оптические параметры

Таблица 21: Метод, геометрия и оптические параметры.

Параметр	Аргументы	Описание
`--method`	`po|go`	Физическая или геометрическая оптика.
`--incoherent`	нет	Складывать матрицы Мюллера отдельных пучков без интерференционных членов.
`--particle`	`TYPE ...`	Создать встроенную частицу.
`--particle-file`	`FILE`	Прочитать геометрию из файла.
`--geometry`	`auto`, `convex` или `nonconvex`	Автоматически определить или явно задать класс геометрии.
`--save-geometry`	`FILE`	Сохранить фактически использованную геометрию.
`--resize-dmax-um`	`DMAX`	Масштабировать файловую частицу по $D_{\max}$, мкм.
`--k-eq`	`K`	Масштабировать по эквивалентному объёмному параметру.
`--refractive-index`	`REAL IMAG`	Комплексный показатель преломления.
`--wavelength-um`	`LAMBDA`	Длина волны, мкм.

Параметр	Аргументы	Описание
`--max-reflections`	`N`	Максимальная глубина внутренних событий, от 0 до 30.
`--absorption`	нет	Явно включить учёт поглощения; ненулевая мнимая часть также включает его.

18.2. Ориентации и адаптивная сходимость

Таблица 22: Основные способы задания ориентаций.

Параметр	Аргументы	Описание
`--fixed-orientation`	`BETA GAMMA`	Одна ориентация в градусах.
`--orientation-file`	`FILE`	Прочитать пары β, γ в градусах из файла.
`--parallel-trace`	нет	Параллельная трассировка CPU PO из файла; потоки задаёт `--threads`.
`--deduplicate-orientations`	нет	Одна трассировка точных повторов с суммарными весами.
`--analytic-backscatter`	нет	Аналитический компонент и полный остаток $M_{11}(180^\circ)$.
`--analytic-return-order`	K	Возвратные self-семейства: 1–8, default 1.
`--euler-grid`	`NBETA NGAMMA`	Регулярная сетка $\beta \times \gamma$.
`--euler-quadrature`	`NBETA NGAMMA`	Квадратура Гаусса по $\cos\beta$ и периодическая сетка по γ.
`--diffraction-sampling`	`Q`	Задать шаг ξ/Q одновременно для ориентаций и рассеяния.
`--orientation-diffraction-sampling`	`Q`	Задать или переопределить шаг β, γ как ξ/Q.
`--sobol`	`N`	Квазислучайная последовательность Соболя.
`--so3-quaternion`	`N`	SO(3) с сокращением по симметрии; α интегрируется по ϕ.
`--so3-mirror-audit`	нет	Явные вложенные отражённые пары для проверки `--mirror-gamma`; только чётное N.
`--so3-full-quaternion`	`N`	Прямая кватернионная проверка полной группы $SO(3)$.
`--adaptive-orientations`	`EPS`	Подобрать только число ориентаций Соболя.

Параметр	Аргументы	Описание
`--adaptive-euler-grid`	`EPS`	Раздельно и совместно уточнить N_β и N_γ.
`--auto`	`EPS`	Подобрать N_ϕ, θ и число ориентаций при фиксированном n.
`--autofull`	`EPS`	Дополнительно подобрать глубину n.
`--diffraction-autofull`	`EPS`	Совместный поиск с итоговой регулярной квадратурой Эйлера.
`--adaptive-config`	`FILE`	Прочитать отдельные пределы и допуски из файла.
`--orientation-chunk`	`N`	Ограничить число ориентаций в одном блоке памяти.
`--pole`	нет	На полюсе считать одно значение γ с полным весом.

18.3. Сетка рассеяния и вычислительная платформа

Таблица 23: Угловая сетка и выполнение расчёта.

Параметр	Аргументы	Описание
`--scattering-grid`	`T0 T1 NPHI NTH`	Равномерная сетка; записывается $NTH + 1$ строк.
`--theta-grid-file`	`FILE`	Неравномерная сетка θ из файла.
`--auto-theta-grid`	`EPS`	Адаптивно уточнить интервалы θ.
`--scattering-diffraction-sampling`	`Q`	Задать или переопределить шаги θ и экваториального ϕ как ξ/Q.
`--latitude-phi-grid`	нет	Использовать $N_\phi(\theta) \propto \sin\theta$.
`--adaptive-phi`	`EPS`	Подобрать N_ϕ, представляющее усреднение по α.
`--phi-points`	`N`	Явно зафиксировать N_ϕ.
`--backend`	`auto|cpu|cuda`	Выбрать вычислительную платформу.
`--threads`	`N`	Число потоков OpenMP на ведущем процессе.
`--fft-factor`	`N`	Рассчитать примерно в N раз меньше прямых точек ϕ и интерполировать их.
`--fft-tolerance`	`EPS`	Выполнить одно вложенное уточнение FFT-сетки.

18.4. Отсечения и серии размеров

Таблица 24: Управление стоимостью и сериями расчётов.

Параметр	Аргументы	Описание
`--cutoff-profile`	`off`, `safe`, `balanced`, `fast`	Согласованный набор порогов; не смешивать с отдельными порогами.
`--beam-cutoff-importance`	`EPS`	Отсечение выходного пучка по величине $\|J\|^2 A$.
`--trace-cutoff-importance`	`EPS`	Отсечение внутренней ветви по величине $\|J\|^2 A$.
`--beam-area-ratio`	`R`	Порог удаления малых фрагментов; профиль `off` отключает старое значение 100.
`--trace-max-beams`	`N`	Аварийно завершить расчёт после N узлов дерева; 0 отключает предел.
`--gpu-trace-prefilter`	нет	Явно включить автоматическое пакетное упорядочивание и отбор кандидатов на CUDA.
`--no-gpu-trace-prefilter`	нет	Отключить автоматическую CUDA-трассировку для контрольного сравнения.
`--dmax-grid`	`DMIN DMAX N`	Логарифмическая серия по $D_{\max}$.
`--k-eq-grid`	`KMIN KMAX N`	Логарифмическая серия по k_{eq}.
`--k-eq-list`	`FILE`	Точный список значений k_{eq}.
`--scan-jobs`	`N`	Число дочерних процессов для серии размеров.
`--gpu-devices`	`LIST`	Список GPU для дочерних процессов серии.

18.5. Результаты и диагностика

Таблица 25: Вывод и служебные файлы.

Параметр	Аргументы	Описание
`--output`	`PATH`	Каталог и префикс результата.
`--progress-interval`	`SECONDS`	Период сообщений о ходе расчёта.
`--save-betas`	нет	Сохранять промежуточные матрицы для каждого кольца β.

Параметр	Аргументы	Описание
`--checkpoint`	нет	Включить сохранение и продолжение поддерживаемых длинных расчётов.
`--jones-output`	нет	Записывать матрицы Джонса в поддерживаемых режимах.
`--no-shadow-output`	нет	Дополнительно записать результат без теневого пучка.
`--close`	нет	Завершить процесс после расчёта; сохранён для совместимости.
`--help`	нет	Показать поддерживаемые рабочие параметры.
`--help-debug`	нет	Показать также устаревшие, диагностические и экспериментальные параметры.

19. Выходные файлы и интегральные величины

Основной файл содержит угол θ, вес элемента телесного угла и элементы матрицы Мюллера. В файле они обозначены $M11, \ldots, M44$, что соответствует $M_{00}, \ldots, M_{33}$ в нулевой индексации кода.

Таблица 26: Основные выходные объекты.

Объект	Смысл
`theta`	Угол рассеяния в градусах.
`2pi*dcos`	Вес кольца телесного угла для данной строки.
`Mij`	Элементы Мюллера после суммирования пучков и ориентаций.
`*_noshadow`	Результат без теневого пучка при `--no-shadow-output`.
`*_betas/`	Промежуточные результаты по кольцам β при `--save-betas`.
`checkpoint`	Файлы продолжения поддерживаемых длинных расчётов.
`FILE` из `--save-geometry`	Фактически использованная после масштабирования рабочая геометрия по явно выбранному пути.
`<result>_integrals.tsv`	Машиночитаемая таблица сечений экстинкции, рассеяния и поглощения, эффективностей и оценки ошибки C_{sca}.

В журнал и TSV выводится по одному определению каждой физической величины для

выбранного метода:

$$C_{\text{ext}}, \quad C_{\text{sca}}, \quad C_{\text{abs}}, \qquad Q_{\text{ext}}, \quad Q_{\text{sca}}, \quad Q_{\text{abs}}, \qquad Q_x = \frac{C_x}{A_{\text{proj}}}.$$

В физической оптике C_{ext} вычисляется по оптической теореме из амплитуды рассеяния вперёд. Для поглощающего материала $C_{\text{sca}} = \sum_j M_{11}(\theta_j)\Delta\Omega_j$ и $C_{\text{abs}} = C_{\text{ext}} - C_{\text{sca}}$. Для непоглощающего материала выполняется $C_{\text{sca}} = C_{\text{ext}}$ и $C_{\text{abs}} = 0$. В GO используется $C_{\text{ext}} = A_{\text{proj}}$; при поглощении C_{sca} равен полной энергии выходных пучков, а C_{abs} — их разности. В непоглощающей геометрической оптике снова принудительно выполняется физический баланс. Несогласованные старые определения и величины другого метода в отчёт не добавляются.

Оценка точности интеграла C_{sca} сравнивает полную сетку θ с вложенной сеткой из каждой второй строки:

$$\hat{\epsilon}_{\text{sca}} = \frac{|C_{\text{sca},h} - C_{\text{sca},2h}|}{\max(|C_{\text{sca},h}|, |C_{\text{sca},2h}|)}.$$

Для непоглощающего расчёта берётся максимум этой оценки и независимой невязки баланса: интеграл M_{11} сравнивается с оптической теоремой в физической оптике, а сумма выходных пучков — с площадью проекции в геометрической. При числе строк θ меньше пяти оценка недоступна. Значение больше 1% даёт статус `check_csca_integration`. Это апостериорная проверка сетки, а не строгая граница ошибки и не проверка сходимости по ориентациям.

TSV содержит поля `method`, `label`, `status`, шесть значений C/Q и `Csca_error_estimate_rel`; альтернативных старых определений в нём нет.

В начале и конце журнала записываются имя узла, модель CPU, число логических и физических ядер, фактическое число потоков OpenMP, общий и доступный объём ОЗУ, текущий и максимальный RSS процесса, видимые GPU и память активного устройства. В конце также находятся отчёты FFT и отсечений. Время на занятом другим процессом GPU нельзя сравнивать с измерением на свободном устройстве.

Обычный запуск не создает диагностические файлы в текущем рабочем каталоге. Первый набор PO-пучков записывается только при явно заданном `MBS_BEAM_LOG=PATH`.

Часть VI

Точность, проверка и воспроизводимость

20. Обязательные проверки результата

1. Для сравнения CPU/GPU используйте один бинарный файл и меняйте только `--backend cpu|cuda`.
2. Для направлений 0° и 180° используйте `--pole` и двойную точность. Сетка `--scattering-grid` должна совпадать.
3. При измерении скорости фиксируйте платформу, точность и число GPU. Отдельно записывайте `MBS_GPU_FUSED_MUELLER` и `MBS_GPU_STAGE_MUELLER`.
4. После изменения коэффициентов Френеля, `RotateJones` или интеграла по рёбрам проверяйте не только M_{11}, но и M_{12}, M_{33}, M_{34}, M_{44}.

5. Для невыпуклой частицы или агрегата сравнивайте небольшие расчёты с `--trace-prefilter` и без него: результаты должны совпадать в пределах ошибки округления.

21. Оценка погрешности и выбор параметров

21.1. Нормировка интенсивности

В коде амплитуда пучка и дифракционный интеграл разделены: трассировка даёт $\mathbf{J}_{\mathrm{beam}}$, а обработчик физической оптики домножает её на F_{edge}. Наблюдаемая интенсивность одного направления определяется совместно геометрией апертуры, фазовым интегралом, множителями Френеля, оптическим путём и общей нормировкой:

$$I(\theta,\phi) \propto \left\| \sum_{b=1}^{N_b} C_{\mathrm{norm}}\, F_{\mathrm{edge},b}\, \mathbf{R}_{\mathrm{out},b}\, \mathbf{F}_{n,b} \right\|_F^2 .$$

Здесь $\|\cdot\|_F$ — норма Фробениуса. При сравнении CPU и GPU надо сопоставлять весь вычислительный путь, а не только F_{edge}: части фазы хранятся в `J_phased`, `dirPhase` и накопленной матрице Джонса.

Таблица 27: Компоненты нормировки и где они могут проявляться.

Компонент	Практический смысл
C_{norm}	Общий множитель результата; не должен зависеть от выбранной платформы CPU или GPU.
F_{edge}	Дифракция апертуры; чувствительна к пределу в направлении вперёд.
$\mathbf{R}_{\mathrm{out}}$	Поляризационный поворот; чувствителен к вырождению базиса рассеяния.
$\mathbf{F}_n$	Накопленная матрица Джонса и фаза оптического пути.
w_i	Вес ориентации; особенно важен при `--pole`.

21.2. Устойчивый предел интеграла по рёбрам

Проблемный случай интеграла по рёбрам возникает, когда

$$|A_q| \ll 1, \qquad |B_q| \ll 1.$$

Тогда разность экспонент

$$\exp(\mathrm{i}\Phi_{j+1}) - \exp(\mathrm{i}\Phi_j)$$

вычитает почти равные числа. Устойчивая запись использует

$$\exp(\mathrm{i}u) - \exp(\mathrm{i}v) = \exp\!\left(\mathrm{i}\frac{u+v}{2}\right) 2\mathrm{i}\sin\!\left(\frac{u-v}{2}\right).$$

Если $\delta = (u - v)/2$ мал, то

$$\sin\delta \approx \delta - \frac{\delta^3}{6} + \frac{\delta^5}{120}.$$

Эта запись объясняет, почему направление точно вперёд должно обрабатываться отдельной ветвью: общая формула математически верна, но численно теряет значащие цифры. Для 0° и 180° такая потеря может выглядеть как различие CPU и GPU даже в двойной точности, если одна реализация использовала предельную формулу, а другая осталась в общей ветви.

21.3. Предел базиса Джонса на полюсах

Базис рассеяния зависит от

$$\mathbf{v}_t = \frac{\mathbf{v}_f \times \mathbf{s}}{\|\mathbf{v}_f \times \mathbf{s}\|}.$$

Если $\mathbf{s}$ параллелен $\mathbf{v}_f$, знаменатель равен нулю. При точном рассеянии вперёд или назад азимутальный базис не единственен. После соответствующего поворота базиса Стокса физическая матрица Мюллера должна быть инвариантна к этому выбору:

$$\mathbf{M}_\chi = \mathbf{R}_M(\chi)\,\mathbf{M}_0\,\mathbf{R}_M(-\chi).$$

При усреднении случайных ориентаций многие азимутальные зависимости исчезают, но для одной ориентации они сохраняются. Поэтому отдельно проверяют совпадение M_{11} как интенсивности и совпадение поляризационных элементов в одной конвенции базиса.

21.4. Симметрия матрицы Мюллера вперёд и назад

Для случайных ориентаций и полной азимутальной симметрии рассеяние вперёд удовлетворяет ограничениям

$$M_{12} = M_{21} = M_{13} = M_{31} = 0,$$

а блок Q/U становится диагонально-симметричным в стандартном базисе. Ограничения для обратного направления зависят от конвенции параметров Стокса и реализации поворота на 180°. Помимо алгебраической симметрии надо проверять непрерывность угловых зависимостей:

$$\lim_{\theta\to 0^+} M_{ij}(\theta) = M_{ij}(0), \qquad \lim_{\theta\to \pi^-} M_{ij}(\theta) = M_{ij}(\pi).$$

21.5. Ошибка ориентационного усреднения

Для метода Монте-Карло среднеквадратичная ошибка обычно имеет вид

$$\mathbb{E}\left[\left|\overline{M}_N - \overline{M}\right|^2\right]^{1/2} \approx \frac{\sigma_M}{\sqrt{N}}.$$

Для последовательностей Соболя и Хаммерсли с малой дискрепантностью в гладких задачах часто наблюдается более быстрое убывание:

$$\left|\overline{M}_N - \overline{M}\right| \lesssim C\frac{(\log N)^d}{N},$$

Однако обрезка полигонов, каустики и предельные ветви делают интегранд негладким. Поэтому программа оценивает сходимость по реально вычисленным матрицам, а не по теоретической асимптотике.

21.6. Практическая формула выбора N_{ori}

Если известна грубая относительная вариация V требуемого элемента Мюллера, начальное число ориентаций для метода Монте-Карло можно оценить как

$$N_{\text{ori},0} = \left\lceil \left(\frac{V}{\varepsilon_{\text{target}}} \right)^2 \right\rceil$$

Для регулярной дифракционной сетки начальную оценку получают из углового масштаба:

$$N_\beta \approx \frac{\beta_{\max} - \beta_{\min}}{\Delta\theta_{\text{diff}}/\texttt{DIV}}, \qquad N_\gamma \approx \frac{\gamma_{\max} - \gamma_{\min}}{\Delta\theta_{\text{diff}}/\texttt{DIV}}.$$

Итог:

$$N_{\text{ori}} \approx N_\beta N_\gamma,$$

с поправками на симметрию и `--pole`. Если симметрия частицы уменьшает область γ в s_γ раз, то

$$N_\gamma \leftarrow \left\lceil \frac{N_\gamma}{s_\gamma} \right\rceil.$$

21.7. Ошибка сетки по θ

Сетка θ должна разрешать самую узкую физическую особенность. Для большого размерного параметра x ширина переднего пика имеет порядок

$$\delta\theta \sim \frac{1}{x}.$$

При слишком крупном шаге интегральная интенсивность может быть приемлемой, но высота переднего пика и соседние строки будут неверными. Начальная оценка:

$$\Delta\theta_{\text{front}} \le \frac{1}{5x}\frac{180}{\pi} \quad \text{или строже для контрольного расчёта.}$$

В задней полусфере допустим более крупный шаг, только если поляризационные элементы не имеют быстрых осцилляций. Режим `--auto-theta-grid` распределяет узлы по измеренной ошибке интерполяции.

21.8. Сходимость по глубине трассировки

Параметр `--max-reflections` контролирует число внутренних событий. Если R — характерный модуль коэффициента отражения Френеля, вклад поколения g приближённо убывает как

$$I_g \sim R^{2g}$$

для непоглощающего случая без фокусировок. С поглощением добавляется множитель

$$\exp(-2k\,\mathrm{Im}(m)L_g).$$

Сходимость по глубине проверяют по итоговой матрице Мюллера:

$$\varepsilon_n = \max_r \frac{\|\mathbf{M}_n(r) - \mathbf{M}_{n-1}(r)\|_\infty}{|M_{11,n}(r)| + \varepsilon_0}.$$

Если увеличение глубины слишком дорого, сначала калибруют отсечения пучков и анализируют число реально построенных ветвей.

22. Ускорения и контролируемые приближения

22.1. Отсечение малозначимых пучков

Выходной порог удаляет готовый пучок перед вычислением дифракции, а внутренний порог останавливает ветвь при построении дерева отражений и преломлений. Три безразмерных критерия имеют вид

$$j_b = \frac{\|\mathbf{J}_b\|_F^2}{J_{\mathrm{ref}}^2} < \varepsilon_J, \qquad a_b = \frac{A_b}{A_{\mathrm{ref}}} < \varepsilon_A, \qquad i_b = j_b a_b < \varepsilon_I.$$

Здесь $\mathbf{J}_b$ — накопленная матрица Джонса ветви, а A_b — площадь полигона пучка. Если включено несколько отдельных критериев, используется логическое «или»: достаточно одного условия. Поэтому счётчики причин в журнале могут перекрываться.

22.1.1. Единые профили

`--cutoff-profile MODE` нельзя смешивать с отдельными порогами отсечения. Он также несовместим с `--beam-area-ratio`: конфликт обнаруживается до начала расчета.

Таблица 28: Значения согласованных профилей отсечения.

Профиль	Выход i_b	Трасса i_b	Отношение площадей	Назначение
`off`	0	0	выключено	Эталон, сходимость и поиск ошибок.
`safe`	10^{-10}	10^{-12}	10^8	Первый рабочий кандидат после сравнения с `off`.
`balanced`	10^{-8}	10^{-10}	10^5	Массовые серии после калибровки данного класса частиц.
`fast`	10^{-6}	10^{-8}	10^3	Предварительные расчёты, не контрольные.

Профиль	Выход i_b	Трасса i_b	Отношение площадей	Назначение
Значение по умолчанию	0	$j_b = 10^{-7}$	100	Масштабно-инвариантная замена старого порога, если профиль и отдельные пороги не указаны.

Профили `safe`, `balanced` и `fast` используют совместную важность $|J|^2 A$, а не независимые пороги по амплитуде и площади. Поэтому большая слабая или малая интенсивная ветвь не удаляется только по одному множителю. По умолчанию ветвь также завершается при $|J|^2 < 10^{-7}|J|^2_{\max}$; это относительная замена прежнего порога, который ошибочно зависел от площади частицы. Сохраняется упрощение с отношением площадей 100. Только `--cutoff-profile off` полностью отключает это приближение. Профили не задают `--trace-max-beams`. Для автоматических запусков глубоко вогнутых частиц следует задавать конечный предел как диагностическую защиту: даже строгий профиль может построить очень большое дерево на отдельной ориентации. Ненулевой счётчик `Configured beam-limit hits` означает неполный результат; перед использованием физических данных необходимо увеличить предел или откалибровать более сильные отсечения.

22.1.2. Отдельные пороги

Таблица 29: Отдельные параметры отсечения.

Флаг	Диапазон	Действие
`--beam-cutoff EPS`	$[0, 1]$	Старый общий порог для j_b и a_b; удаление по «или».
`--beam-cutoff-jones EPS`	$[0, 1]$	Удалить выходной пучок при $j_b <$ EPS.
`--beam-cutoff-area EPS`	$[0, 1]$	Удалить выходной пучок при $a_b <$ EPS.
`--beam-cutoff-importance EPS`	$[0, 1]$	Удалить выходной пучок при $i_b <$ EPS.
`--trace-cutoff EPS`	$[0, 1]$	Старый общий порог внутренней ветви для j_b и a_b.
`--trace-cutoff-jones EPS`	$[0, 1]$	Остановить ветвь при $j_b <$ EPS.
`--trace-cutoff-area EPS`	$[0, 1]$	Остановить ветвь при $a_b <$ EPS.
`--trace-cutoff-importance EPS`	$[0, 1]$	Остановить ветвь при $i_b <$ EPS.
`--beam-area-ratio R`	$R \geq 1$	После невыпуклого разбиения оставить больший фрагмент при отношении площадей не меньше R.
`--trace-max-beams N`	$N \geq 0$	Жёсткий предел дерева одной ориентации; 0 отключает его. Достижение предела аварийно завершает расчёт, чтобы неполная ориентация не попала в среднее.

Старый общий параметр можно сочетать с отдельным порогом Джонса или площади: отдельное значение имеет приоритет, а программа выдаёт предупреждение. Для согласованного профиля частичное переопределение запрещено.

Отчёты `OUTPUT BEAM CUTOFF REPORT` и `TRACE CUTOFF REPORT` показывают фактические пороги, число кандидатов, сохранённых и удалённых ветвей, причины и достижения пределов дерева. При калибровке для `off` и выбранного профиля фиксируют ориентации, n, θ и N_ϕ и сравнивают все 16 отношений M_{ij}/M_{11}. Доля удалённых пучков сама по себе не оценивает ошибку из-за когерентной интерференции.

22.2. Двойная и одинарная точность

Для одинарной точности машинное эпсилон имеет порядок

$$\varepsilon_{\text{float}} \approx 1.2 \cdot 10^{-7},$$

для двойной точности

$$\varepsilon_{\text{double}} \approx 2.2 \cdot 10^{-16}.$$

Если суммируется N вкладов похожей величины, ошибка порядка сложения грубо может расти как

$$\varepsilon_{\text{sum}} \sim N\varepsilon_{\text{mach}}$$

при неблагоприятном порядке сложения. Поэтому атомарное накопление FP32 может отличаться от CPU сильнее, чем FP64. Контроль около 0° и 180° следует выполнять сборкой FP64 без быстрых математических функций. Варианты `double_fast` и `float_fast` используют только после калибровки.

22.3. FFT-интерполяция по ϕ

Пусть N_ϕ — число азимутальных точек результата, а f — коэффициент сжатия. Первый проход CUDA непосредственно вычисляет примерно

$$N_{\phi,\text{direct}} = \max\left(16, \left\lfloor \frac{N_\phi}{f} \right\rfloor\right)$$

точек. Затем периодическое дополнение спектра Фурье нулями восстанавливает полную сетку. При $f = 1$, $N_\phi < 32$ или отсутствии фактического уменьшения сохраняется прямой путь без интерполяции.

`--fft` включает автоматический выбор f. Флаг `--fft-factor N`, $1 \le N \le 64$, явно задает степень сжатия и имеет приоритет над старой переменной окружения `MBS_FFT_PHI_FACTOR`. Восстановление предполагает ограниченный спектр:

$$f(\phi) = \sum_{m=-M}^{M} c_m e^{\mathrm{i}m\phi}.$$

Если существенны гармоники $|m| > M$, возникает наложение спектров или сглаживание. Большой коэффициент сжатия сам по себе не гарантирует точность.

`--fft-tolerance EPS`, $0 < \mathrm{EPS} < 1$, добавляет вторую прямую сетку

$$N_{\phi,2} = \min(N_\phi, 2N_{\phi,\mathrm{direct}}).$$

Код сравнивает значимые M_{11} и все 16 элементов Мюллера в масштабе M_{11}:

$$\hat{e} = \max\left(\max_{M_{11}\ \text{significant}} \frac{|M_{11}^{(1)} - M_{11}^{(2)}|}{|M_{11}^{(2)}|}, \max_{i,j} \frac{|M_{ij}^{(1)} - M_{ij}^{(2)}|}{\max(|M_{11}^{(2)}|, 10^{-3} M_{11,\max}^{(2)})} \right).$$

Если $\hat{e} > \mathrm{EPS}$, сохраняется результат с удвоенным числом прямых точек ϕ. Это одно апостериорное уточнение, а не строгая верхняя граница ошибки и не итерационная проверка сходимости. Оно требует дополнительного дифракционного прохода и одновременно хранит грубый, уточнённый и выходной массивы, увеличивая время и максимальную память.

```
# Сжатие в 4 раза; одно уточнение при ошибке выше 1%
gpu/bin/mbs_po_gpu_float_fast ... --fft-factor 4 --fft-tolerance 0.01
```

Отчёт `FFT INTERPOLATION REPORT` содержит запрошенный и фактический коэффициенты, прямое и выходное N_ϕ, число проходов FFT, проверок и уточнений, а также наихудшую оценку. Значение $f = 1$ явно обозначает прямой расчёт. Для эталона параметры FFT убирают или задают `--fft-factor 1`; рабочее сжатие проверяют на характерных формах и размерах. Совмещённый путь нескольких k_{eq} не выполняет вложенную проверку: при заданном допуске диспетчер выбирает поддерживаемый прямой путь.

22.4. Повторное использование трассировки для нескольких размеров

Для ряда размеров геометрическое дерево пучков может совпадать, а фаза изменяется через kD:

$$\Phi_j(k, D) = kD\,\xi_j,$$

где ξ_j — безразмерная проекция вершины. Тогда можно повторно использовать нормированные вершины и коэффициенты рёбер:

$$x_j(D) = D\,\hat{x}_j, \qquad y_j(D) = D\,\hat{y}_j.$$

Это ускоряет серию размеров, но требует контроля: если размер меняет пороги или автоматически выбранные сетки, повторное использование может перестать быть эквивалентным полной пересборке.

23. Диагностика и воспроизводимость

23.1. Поиск расхождения CPU/GPU

Если найдено отличие между CPU и GPU, его надо локализовать сверху вниз:

1. Использовать один бинарный файл и переключать только `--backend cpu|cuda`. Так исключаются различия интерфейса и настроек ведущего кода.

2. Зафиксировать двойную точность, `--scattering-grid`, `--fixed-orientation` и `--max-reflections`. На первом проходе отключить FFT и отсечения.
3. Проверить M_{11} отдельно от поляризационных элементов. Если отличается только поляризационный блок, вероятна ошибка поворота базиса.
4. Сравнить результат с `--pole` и без него. Отличие только в крайних направлениях указывает на полюсный вес или вырожденный базис.
5. Отключить совмещённое и поэтапное накопление, затем сравнить пути с атомарными операциями и без них. Различие младших разрядов связано с порядком редукции; отличие в процентах означает разные ветви алгоритма.
6. Если одна ориентация совпадает, а среднее нет, проверить веса, границы фундаментальной области и разбиение ориентаций на блоки.

23.2. Выбор параметров итогового расчёта

Параметры подбирают последовательно, изменяя на каждом шаге одну величину:

1. На небольшом размере подобрать `--max-reflections`, пороги отсечения и сетку рассеяния.
2. На среднем размере сравнить `--orientation-diffraction-sampling 2` и `--orientation-diffraction-sampling 4` при неизменной сетке рассеяния; при заметном отличии продолжить уточнение или использовать адаптивный режим.
3. На нескольких характерных размерах сравнить одинарную и двойную точность, а также сборки с быстрыми математическими функциями и без них.
4. Распределение по нескольким GPU, FFT и повторное использование трассировки включать только после проверки эквивалентного прямого расчёта.
5. Вместе с результатом сохранить полную команду, ревизию Git, версии компилятора, CUDA и драйвера, модель GPU и переменные окружения.

23.3. Проверка геометрии из файла

Точный синтаксис собственного формата приведён в разделе «Внутренний формат частицы и экспорт геометрии». После чтения любая частица представлена набором вершин и упорядоченных граней:

$$\mathcal{P} = \left(\{\mathbf{v}_i\}_{i=1}^{N_v}, \{\mathcal{F}_j\}_{j=1}^{N_f} \right).$$

Грань $\mathcal{F}_j$ является упорядоченным списком индексов вершин:

$$\mathcal{F}_j = (i_{j,1}, i_{j,2}, \dots, i_{j,m_j}).$$

Нормаль грани вычисляется из обхода вершин. Для треугольной части:

$$\mathbf{n}_j = \frac{(\mathbf{v}_{i_2} - \mathbf{v}_{i_1}) \times (\mathbf{v}_{i_3} - \mathbf{v}_{i_1})}{\left\| (\mathbf{v}_{i_2} - \mathbf{v}_{i_1}) \times (\mathbf{v}_{i_3} - \mathbf{v}_{i_1}) \right\|}.$$

Обратный обход меняет знак нормали. Это критично: ветвление по коэффициентам Френеля, определение внутренней стороны и обрезка пучков используют ориентацию нормали. У

невыпуклой частицы грани не должны самопересекаться, а соседние полигоны должны иметь согласованную топологию.

Таблица 30: Проверки геометрии из файла перед расчётом.

Проверка	Зачем нужна
Нет нулевых рёбер	Иначе нормаль и коэффициенты рёбер становятся неопределёнными.
Грань плоская	Апертурный интеграл предполагает плоский полигон.
Нет самопересечения грани	Обрезка и формула площади требуют простого полигона.
Согласованный обход	Нужен правильный знак нормали.
Нет дублированных почти совпадающих граней	Иначе трассировка даёт повторные пересечения и неустойчивую обрезку.
Разумный масштаб	Слишком малые или большие координаты ухудшают обусловленность.

23.4. Площадь, объём и центр

Площадь полигона в локальной плоскости:

$$A_j = \frac{1}{2}\left|\sum_{l=1}^{m_j}(x_l y_{l+1} - x_{l+1} y_l)\right|.$$

Центр грани:

$$\mathbf{c}_j = \frac{1}{m_j}\sum_{l=1}^{m_j}\mathbf{v}_{i_{j,l}}.$$

Объем замкнутого многогранника удобно выражается через тетраэдры с началом координат:

$$V = \frac{1}{6}\sum_{\text{triangles}}\mathbf{a}\cdot(\mathbf{b}\times\mathbf{c}).$$

Знак объёма зависит от ориентации обхода. Отрицательное или близкое к нулю значение для заведомо объёмной частицы почти всегда указывает на ошибку геометрии.

23.5. Проверка чтения выходной строки

Типовая выходная строка содержит угол, вес и 16 элементов матрицы Мюллера:

$$\theta_j, \quad \Delta\Omega_j, \quad M_{11}, M_{12}, \ldots, M_{44}.$$

В терминах кода:

$$M11 \equiv M_{00}, \qquad M44 \equiv M_{33}.$$

Если результат усреднён по ориентациям, каждый элемент уже содержит сумму

$$M_{ab}^{\text{out}}(\theta_j) = \frac{1}{W} \sum_i w_i M_{ab}^{(i)}(\theta_j).$$

Если затем нужна интегральная характеристика по углам, используется вес `2pi*dcos`:

$$\langle M_{ab} \rangle_\Omega = \sum_j M_{ab}^{\text{out}}(\theta_j) \Delta\Omega_j.$$

Не надо повторно умножать на 2π, если колонка уже называется `2pi*dcos`.

23.6. Матрица контрольных тестов

Таблица 31: Минимальная матрица проверок после изменения физического кода.

Тест	Команда/режим	Что должен поймать
Одна ориентация	`--fixed-orientation B G`	Ошибки поворота Джонса, коэффициентов Френеля и интеграла по рёбрам.
Только крайние углы	`--scattering-grid 0 180 N 1 --pole`	Полюсный вес и вырожденный базис.
CPU и GPU одного бинарного файла	`--backend cpu|cuda`	Различия платформ при одинаковом ведущем коде.
Двойная и одинарная точность	Бинарные файлы `double` и `float`	Потери точности и влияние быстрых функций.
Некогерентная сумма	`--incoherent`	Ошибки порядка суммирования Джонса и Мюллера.
Невыпуклая обрезка	Предварительный отбор включён и выключен	Ошибки отбрасывания по проекциям AABB.
Сходимость ориентаций	`--orientation-diffraction -sampling 1, 2, 4`	Недостаточное число ориентаций.
Сходимость глубины	`--max-reflections N` и `N+1`	Недостаточная глубина трассировки.

23.7. Типичные причины больших расхождений

Таблица 32: Диагностика расхождений.

Симптом	Вероятная причина	Что проверить
Отличается только 0°	Полюсная ветвь	`--pole`, границы ячейки θ, предельный базис.

Симптом	Вероятная причина	Что проверить
Отличается только 180°	Обратный базис или знак Френеля	`--legacy-sign`, `BackwardScattering`, поворот Мюллера.
M_{11} совпадает, M_{33} и M_{34} нет	Соглашение о поляризационном базисе	`RotateJones` и экспериментальная ветвь Карчевского.
Одинарная точность заметно отличается	Компенсация больших вкладов или быстрые функции	Околопередние строки и порядок атомарной редукции.
GPU отличается только на большой сетке	Порядок редукции или разбиение на блоки	Совмещённый и поэтапный пути, размер блока.
Невыпуклый результат нестабилен	Ошибка кандидатов на обрезку	Предварительный отбор, нормали граней, самопересечения.
Автоматический и ручной режимы различаются	Разные сетки углов и ориентаций	Сохранить выбранные сетки и повторить их явно.

23.8. Контроль интегрального баланса

Для непоглощающих частиц необходимо контролировать интегральные величины, хотя приближения физической и геометрической оптики не обязаны выполнять точный баланс на любой конечной сетке. Если $S_{11}(\theta)$ соответствует M_{11}, численная величина рассеяния равна

$$C_{\mathrm{sca,num}} = \sum_j S_{11}(\theta_j)\Delta\Omega_j.$$

При уточнении сеток θ и ориентаций эта величина должна стабилизироваться. Скачки при небольшом изменении N_θ указывают на неразрешённый передний пик.

23.9. Минимальные данные для воспроизводимого отчета

Вместе с результатом сохраняют:

Таблица 33: Данные для воспроизводимости.

Данные	Почему нужны
Ревизия Git	Физические формулы и реализация CUDA могли измениться.
Полная команда	Значения по умолчанию могут различаться между версиями.
Компилятор и `ARCH_FLAGS`	Векторизация и слияние операций зависят от сборки.
Версии CUDA и драйвера	Они определяют реализацию библиотек и математических функций.
Модель GPU	Скорость FP32, FP64 и атомарных операций различается.

Данные	Почему нужны
Переменные окружения	Через них выбираются совмещённый, поэтапный и многокарточный режимы.
Хеш файла частицы	Геометрия должна совпадать побайтно.

Часть VII

Приложения

A. Связь с файлами кода

Таблица 34: Где искать описанные объекты в коде.

Файл/область	Что там находится
`src/main.cpp`	Запуск расчёта, адаптивные режимы и справка.
`src/CliOptions.cpp`	Канонические имена параметров, алиасы и группы справки.
`src/RunConfig.cpp`	Проверка совместимости параметров и построение конфигурации запуска.
`src/Splitting.cpp`	Коэффициенты Френеля, направления отражённой и прошедшей ветвей, оптический путь.
`src/Beam.*`	Геометрия пучка, матрица Джонса и состояние полигона.
`src/scattering`	Пересечения луча с гранью и обрезка для выпуклых и невыпуклых частиц.
`src/handler/Handler.cpp`	Скалярные и векторизованные интегралы дифракции по рёбрам.
`src/handler/HandlerPO.cpp`	Поворот Джонса, дифракция, накопление Мюллера и пути AVX.
`src/math/Mueller.cpp`	Преобразование Джонса в Мюллера и повороты матрицы Мюллера.
`src/cuda` и файлы `*.cu`	Ядра CUDA, редукции и совмещённые вычислительные пути.

B. Контрольные формулы для отчётов

B.1. Относительная ошибка элемента Мюллера

Для сравнения двух расчетов A и B удобно печатать относительную ошибку по каждому элементу:

$$\delta_{ij}(\theta) = \frac{M_{ij}^{A}(\theta) - M_{ij}^{B}(\theta)}{|M_{11}^{A}(\theta)| + \varepsilon_0}.$$

Нормировка на M_{11} полезна, потому что слабые поляризационные элементы могут проходить через ноль. Для абсолютного контроля слабых элементов дополнительно используют

$$\Delta_{ij}(\theta) = M_{ij}^{A}(\theta) - M_{ij}^{B}(\theta).$$

В отчетах лучше указывать обе величины: относительная ошибка показывает физический масштаб, абсолютная ошибка показывает численный шум около нулей.

B.2. Интегральная ошибка по углам

Если сравниваются полные угловые зависимости, используется взвешенная среднеквадратичная ошибка:

$$E_{ij}^{2} = \frac{\sum_j \left(M_{ij}^{A}(\theta_j) - M_{ij}^{B}(\theta_j)\right)^2 \Delta\Omega_j}{\sum_j \left(|M_{11}^{A}(\theta_j)| + \varepsilon_0\right)^2 \Delta\Omega_j}.$$

Такая метрика не определяется одной точкой, но направления 0° и 180° всё равно проверяют отдельно: для них используются специальные численные пределы.

B.3. Сравнение времени

Время расчёта раскладывают на компоненты:

$$T_{\text{total}} = T_{\text{trace}} + T_{\text{diffraction}} + T_{\text{reduction}} + T_{\text{io}}.$$

GPU ускоряет прежде всего $T_{\text{diffraction}}$ и часть $T_{\text{reduction}}$. Для сложной невыпуклой частицы может доминировать T_{trace}, поэтому общее ускорение будет ниже. При сравнении CPU и GPU фиксируют

$$N_{\text{ori}}, \quad N_\theta, \quad N_\phi, \quad N_b, \quad n, \quad \text{пороги отсечения.}$$

B.4. Оценка памяти GPU

Грубая оценка памяти для массива матриц Джонса:

$$B_J \approx N_\theta N_\phi N_{\text{ori,chunk}} \cdot 4 \cdot 2 \cdot s_{\text{real}},$$

где 4 — число комплексных элементов Джонса, 2 — действительная и мнимая части, а $s_{\text{real}} = 4$ для одинарной и 8 для двойной точности. Для массива матриц Мюллера

$$B_M \approx N_\theta N_\phi N_{\text{ori,chunk}} \cdot 16 \cdot s_{\text{real}}.$$

Совмещённые ядра уменьшают потребность в полном B_J: сумма Джонса остаётся в регистрах или локальной памяти до преобразования в матрицу Мюллера.

C. Контрольный список перед публикацией результатов

1. Указать ветку и ревизию Git.
2. Указать исполняемый файл CPU/GPU, точность и использование быстрых математических функций.
3. Указать источник частицы: параметры встроенного генератора или хеш файла геометрии.
4. Указать λ, $m = n + \mathrm{i}\kappa$, размер и `--max-reflections`.
5. Указать способ усреднения и фактическое число ориентаций.
6. Указать сетки θ, ϕ и использование `--pole`.
7. Указать пороги отсечения и параметры обрезки невыпуклых частиц.
8. Указать переменные окружения GPU.
9. Приложить сравнение хотя бы двух разрешений угловой и ориентационной сеток.
10. Для быстрого GPU-режима приложить контроль двойной точности на подвыборке.

C.1. Минимальная таблица в статье или отчете

Таблица 35: Минимальные поля таблицы воспроизводимости.

Поле	Пример
Код	`MBS-fast, commit abcdef0.`
Платформа	`gpu/bin/mbs_po_gpu_double.`
Частица	`--particle 1 125.9 78.09` или `--particle-file particle.dat.`
Оптика	`--refractive-index 1.3116 0 --wavelength-um 0.532 --max-reflections 12.`
Ориентации	`--orientation-diffraction-sampling 2 --pole.`
Сетка рассеяния	`--scattering-grid 0 180 600 180.`
Оборудование	Модели CPU и GPU, версии драйвера и CUDA.
Проверка	Максимальная ошибка CPU/GPU, ошибка при уточнении ориентаций и глубины.

C.2. Что не надо смешивать в одном сравнении

Нельзя одновременно менять вычислительную платформу, точность, сетку ориентаций и сетку θ, а затем приписывать всё отличие GPU. В корректной серии каждый переход меняет только один фактор:

$$A_0 \to A_1 \to A_2 \to \cdots,$$

где каждая стрелка меняет только один параметр. Например:

$$\begin{aligned} \text{CPU, двойная точность} &\longrightarrow \text{GPU, двойная точность} \\ &\longrightarrow \text{GPU, одинарная точность} \\ &\longrightarrow \text{GPU, одинарная точность и быстрые функции.} \end{aligned}$$

D. Калибровка и многоуровневое ускорение GPU

Переносимый контроллер `scripts/run_gpu_optimized.py` сочетает уровни глубины 8, 10 и 12; последний без отсечения. Независимый пилот выбирает число азимутов, CUDA-ядро и фиксированные веса аналитических компонентов. Требуются поточечные интервалы и две проверки уточнения. Полная спецификация находится в Markdown-мануале.

```
python3 scripts/run_gpu_optimized.py \
  --binary gpu/bin/mbs_po_gpu_double --case test1 \
  --output results/test1 --calibrate \
  --depths 8 10 12 --level-cutoffs .001 off off \
  --gpus 0 1 2 3 --threads 16
```

Область назад. Режим `--case test3` задаёт призму 316.2 на 123.8 мкм и 227 углов от 170 до 180 градусов. Внешняя theta-сетка сохраняет исходные узлы. Список GPU может содержать любое число разных устройств. Для отдельной поправки слабых пучков используйте:

```
--analytic-controls off --gpus 1 2 3
--depths 8 12 18 18 --level-cutoffs .001 .001 .001 off
--initial-counts 131072 8192 8192 1024
--min-correction-count 1024
```

Последний уровень оценивает 18 отражений без отсечения. При одинаковой глубине cutoff должен строго уменьшаться. Отключение аналитических компонентов сохраняет полный когерентный интеграл и позволяет считать угловые сетки, для которых таблица известного среднего превышает бюджет.